

UAS OneSpan Multi-Device Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS OneSpan Multi-Device Implementation Guide.....	3
2. Introduction to UAS OneSpan Multi-Device Implementation.....	4
2.1. Implementation Planning.....	7
3. OneSpan Multi-Device Token Configuration.....	16
4. UAS OneSpan Multi-Device Implementation Using API.....	20
4.1. Multi-Device Token Activation.....	21
4.2. 2FA Login with OneSpan OTP.....	49
4.3. Standard Verify OTP Only.....	52
4.4. OneSpan Challenge-Response 2FA Login.....	54
4.5. Standard Digital Signature Verification.....	58
4.6. Other Challenge-Response Sample.....	62
4.7. Secure Channel Verification.....	66
4.8. Push Notification Feature.....	72
5. UAS OneSpan Multi-Device Token Common Use Cases.....	76
6. Appendices.....	80

1. Preface - UAS OneSpan Multi-Device Implementation Guide

AccessMatrix Universal Authentication Server (UAS) supports the use of the OneSpan (formerly VASCO) multi-device, which supports multi-device capabilities under a single license.

Intended Reader

This guide is a reference for security administrators, professional service staff and developers.

Document Scope

This document explains the concept of using a OneSpan multi-device instance license and how to integrate and configure OneSpan (formerly VASCO) multi-device tokens with AccessMatrix Universal Authentication Server (UAS). Also, it consists of a complete API integration of OneSpan multi-device token with AccessMatrix UAS.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix UAS Product Suite:

- [AccessMatrix Common Administration Guide](#)
- [UAS Administration Guide](#)
- [AccessMatrix REST API](#)

In the OneSpan documentation, refer to:

- [OneSpan VACMAN Controller Product Guide](#)

2. Introduction to UAS OneSpan Multi-Device Implementation

The single-device solutions generally involve the use of a hardware token generator or a software token generator (one-time-password generation, verification code, etc.) It is, by definition, is unique and assigned to a single user. Multiple device support is directly tied to the use of software token generators, which may be installed on multiple different devices for the same user. These days a user mostly owns and uses multiple devices for bank transactions, identifications, applications access and other security tasks. This section provides the conceptual information to implement OneSpan multi-device instance license with AccessMatrix UAS.

Standard Single Device License and Multi-Device License

OneSpan's DIGIPASS (a brand name for a security token and other authentication products and services) supports multi-device capabilities. The procedure to use a multi-device license is different from one that uses the standard single-device license.

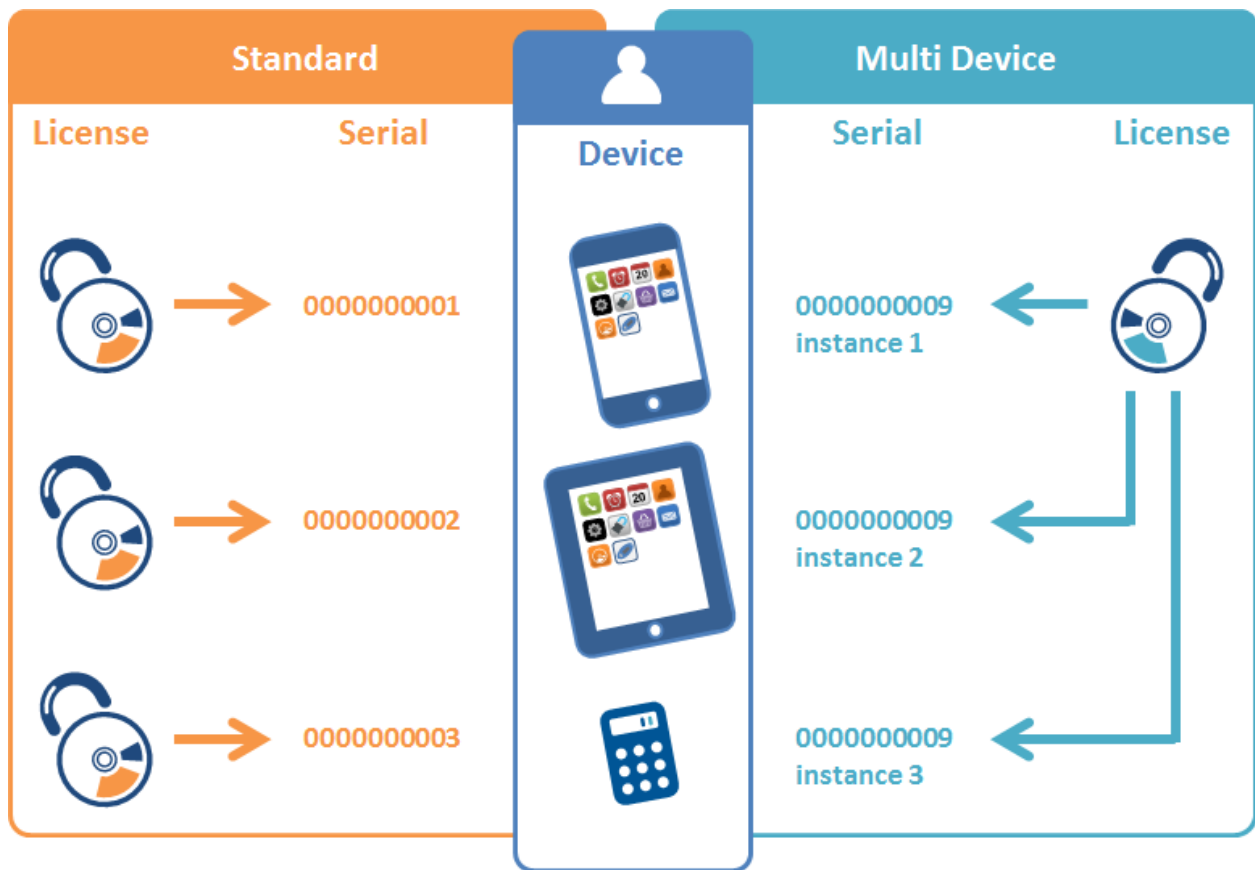
Standard Single Device License Model

A unique 10-digit serial number is generated by OneSpan and associated with DIGIPASS data on the server-side. The DIGIPASS can thus be instantiated on a single device to ensure the symmetry.

Multi-Device License Model

This model supports the single license multi-instances concept. OneSpan generates a unique 10-digit serial number, and this unique serial number can be associated with several DIGIPASS data on the server-side. Each DIGIPASS data is identified by its unique serial number and a sequence number. On the client-side, the DIGIPASS license can thus be instantiated several times. This process fits the deployment of one DIGIPASS per device of the same user. Each DIGIPASS of the user shares the same serial number but has a different sequence number.

Standard Single vs. Multi-Device Licensing Comparison Diagram

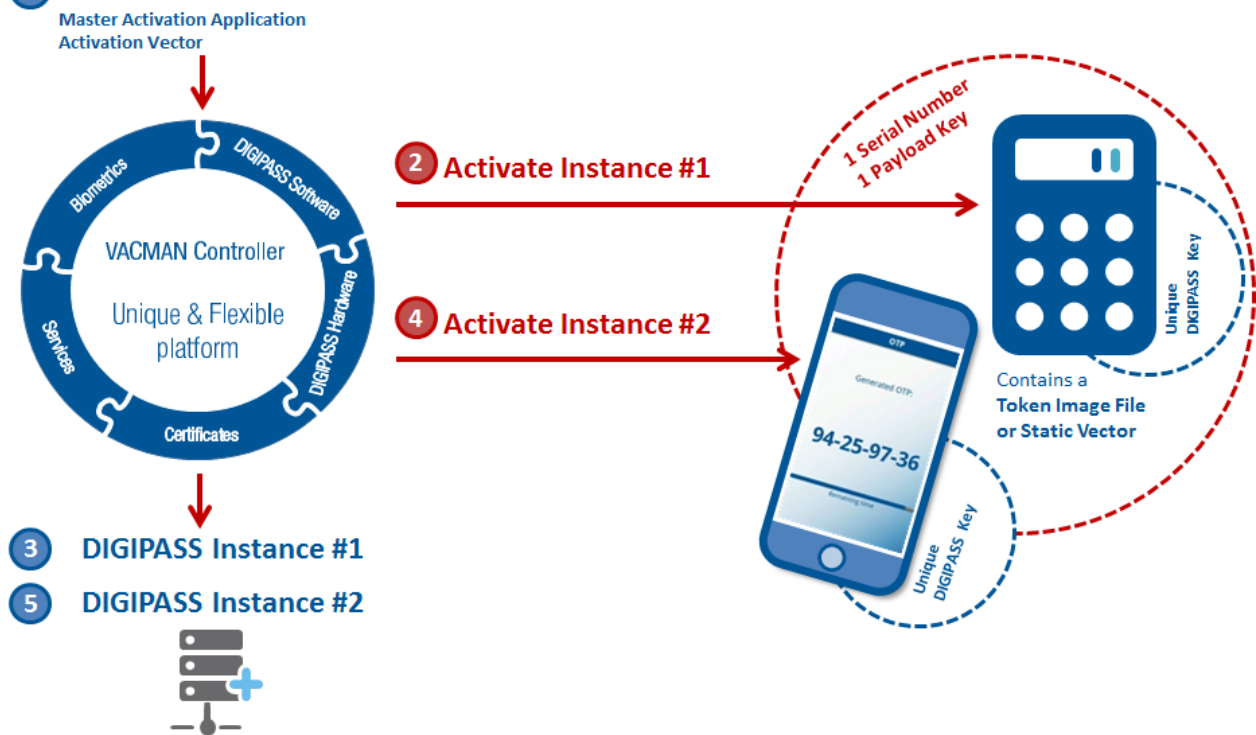


Multi-Device License Types

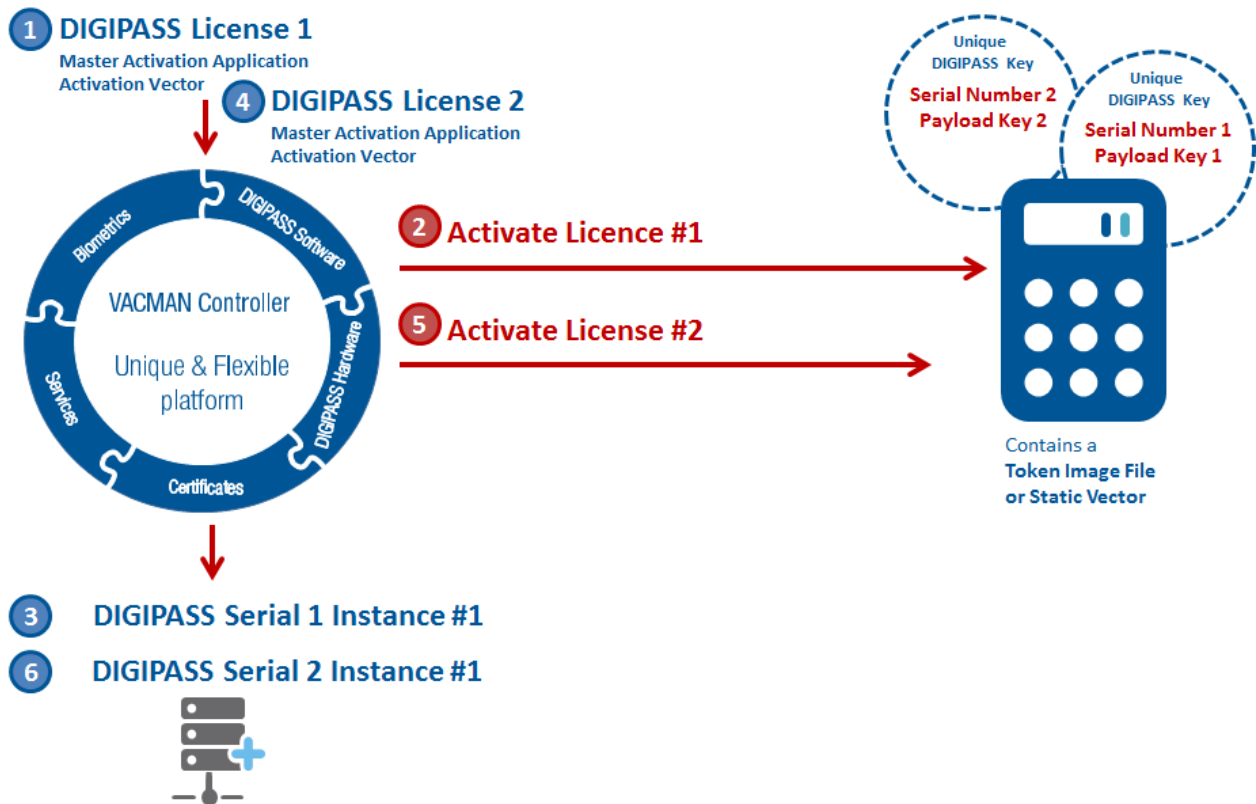
The concept of a single license for multiple devices of a user helps to save cost as compare to multiple licenses for multiple devices of a user. The DIGIPASS data model consists of DIGIPASS serial number, DIGIPASS activation type, DIGIPASS application names, DIGIPASS application authentication modes (RO - Response-Only; CR - Challenge-Response, etc.) and DIGIPASS application blobs.

Single License Multiple Instances

1 DIGIPASS License = 1 Serial Number



Multiple License Multiple Instances



2.1. Implementation Planning

Implementation Checklist

AccessMatrix UAS currently supports the following features.

- **User Authentication via OTP**
 - Using a mobile phone as an OTP token.
 - **User Authentication via Challenge-Response (CR)**
 - Using a mobile phone to generate a response for a challenge from the server.
 - **Transaction Protection via Signature (SIGN)**
 - Using a mobile phone for a transaction signature.
 - Transaction signing via manual entry of data fields.
 - Transaction signing via QR code.
 - **Transaction Signature via Secure Channel**
 - Transaction signature through Secure Channel using CrontoSign.
 - Transaction data will be encrypted by a unique payload key.
 - **Token Time Synchronization**
 - Synchronize token with the server by supplying 2 OTPs.
 - **Token Time Synchronization**
 - Synchronize time between token's application BLOBs.
 - **Online Token Activation**
 - Online activation using Activation Password with data encryption.
 - **Offline Token Activation**
 - 2-step offline activation through CrontoSign.
 - **Token Deactivation**
 - Deactivation through CrontoSign.
 - **Device Binding**
 - Device and token seed are bound upon activation using the unique device id and device type.
-

-
- If the mobile app somehow being duplicated on another phone, the user's security will not be compromised as the server will always reject the verification of a duplicated device.
 - **Push Notification Registration**
 - Register push notification Id (UAS will store the push notification Id, retrievable for use of sending push notification).
 - **Score-Based Response**
 - Score-based DIGIPASS responses (OTP or signature) from the client allows us to obtain information via VACMAN Controller by means of response about the risks evaluated on the client-side. Risk categories: User, Platform, Context.

To use OneSpan DIGIPASS SDK for multi-device implementation with AccessMatrix UAS, you must set up OneSpan Multi-Device token. See below a checklist to configure OneSpan multi-device token.

Item	Tasks
1.	Configure Token Libraries
2.	Configure Token Policy
3.	Configure Token Batch
4.	Configure Token Group
5.	Configure Token Import

Implementation Planning

In any OneSpan VACMAN Controller implementation, there will undoubtedly be a desire to capture and store a variety of details about the authentication state and token information. There are some common areas related to OneSpan token and license data where all implementation will benefit from some best practice, recommendations and considerations.

Security Consideration

Multi-device management allows you to utilize all devices through a single license. How do you ensure all your devices are secure? There are many security measures for your considerations such as secure storage, secure channel, s-signature, transaction signing, QR code support, Cronto authentication and more.

Differences between QR Code and CrontoSign

CRONTO is a patented visual transaction signing solution by OneSpan that enables banks and other financial institutions to counter increasingly sophisticated attacks effectively. You can scan the Cronto code with the camera on your mobile device. Only devices registered with that user's account can decode, decrypt, and display the transaction details for transaction verification.

For multi-device license instances, should you use QR code or CrontoSign for visual authentication and transaction signing verification? See the comparison chart below for QR code and CrontoSign.

QR Code	CrontoSign
Black and White code 	Colour Cronto code 
Support various standards (UPC-A, UPC-E, EAN-8, EAN-13, Code 39, Code 93 and Code 128 etc.)	National Institute of Standards and Technology (NIST)
It carries huge transaction data	The large colour display allows you to present all payment details in one simple view for user verification. It carries reasonably large transaction data
Transaction data up to 7049 characters	Encrypted transaction data. Payload is 75 bytes which is equivalent to 111 encoded character limit (16-bit encoding by triples: 2 bytes = 3 encoded characters)
Reliable	Fast, reliable and versatile

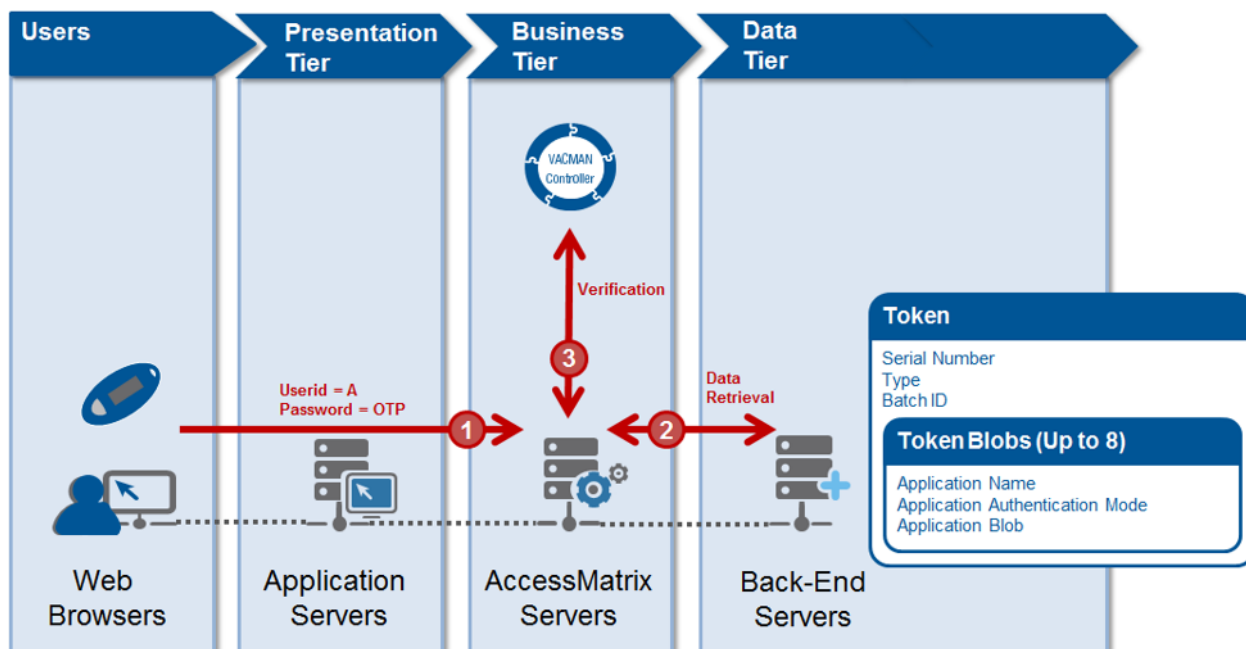
Integration Consideration

AccessMatrix UAS OneSpan Multi-Device Licenses and its Architecture

Single Device Sample Architecture

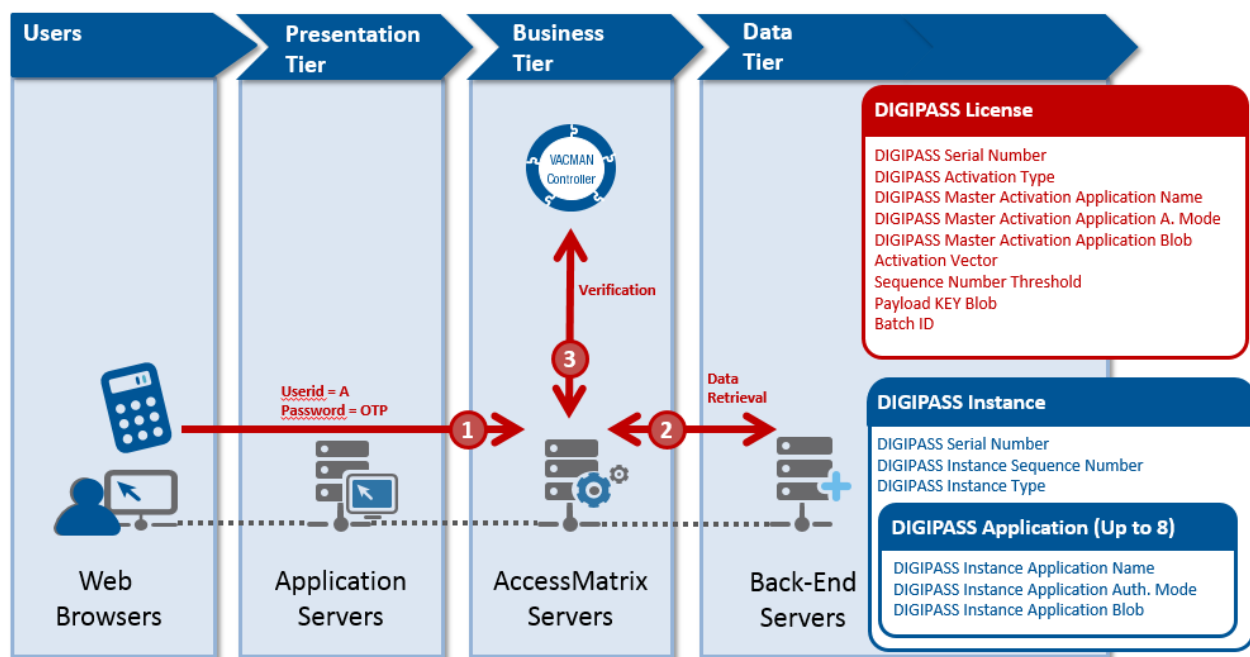
In the diagram below, a user logs in an application using his or her user Id and password. The user's information is sent to the AccessMatrix server for authentication. The token serial number, type and

batch Id will be sent to the data backend server to generate a license. The data will be retrieved and returned in AccessMatrix server for verification (verify by using VACMAN Controller SDK). The authorization code or OTP is returned to the user for usage. This use case is applied to one user, one device with one license on an application. If the user uses two or more devices, there are more licenses.



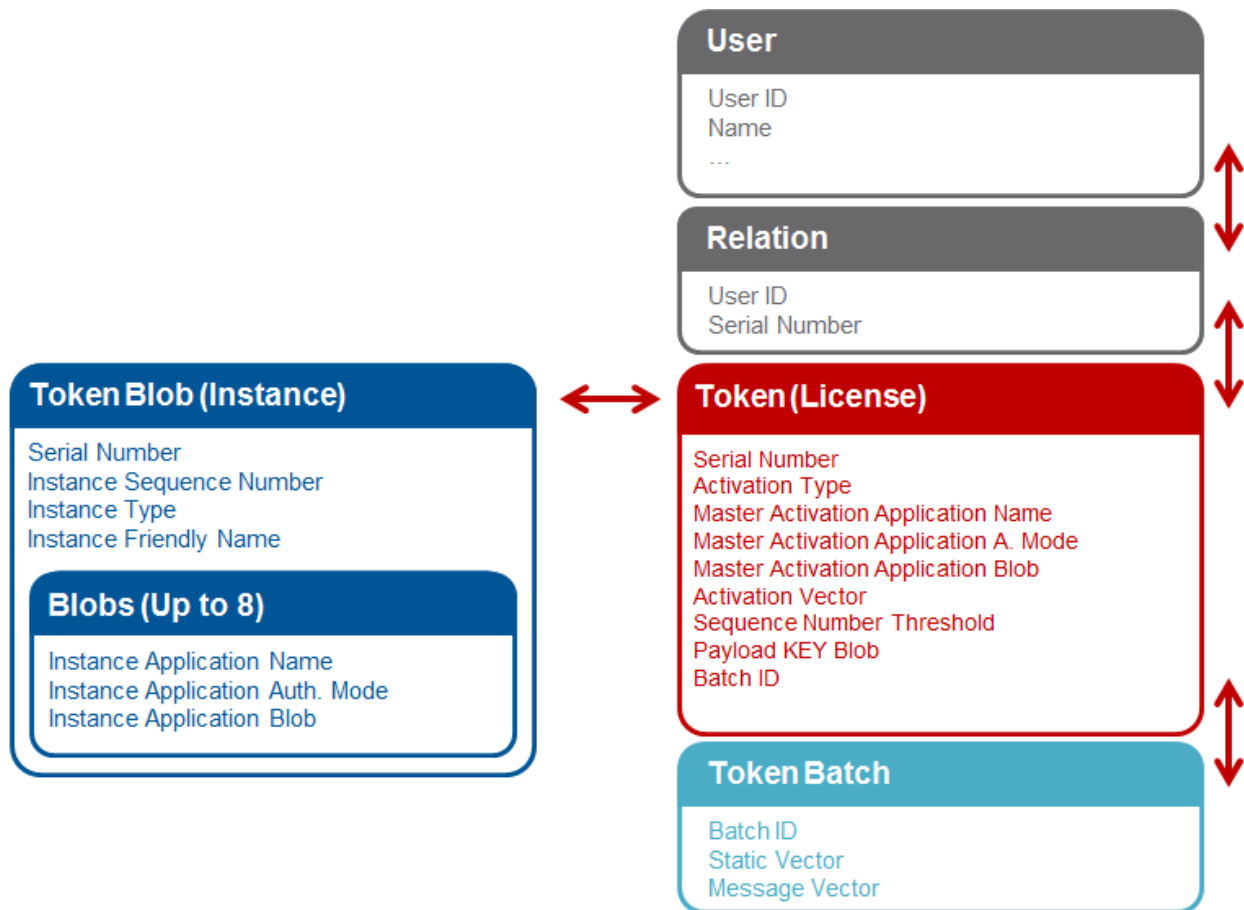
Multi-Device Sample Architecture

The use case in the diagram below is applied to a user who uses multiple devices (i.e. tablet, phone and desktop) for one application with a single license. The procedure is similar to the single device architecture except for the single DIGIPASS license will generate a DIGIPASS sequence number for each device. Therefore if the user has three devices, he or she will have three DIGIPASS sequence numbers that are generated from a license. Each DIGIPASS token can have from one to eight DIGIPASS applications (authentication, e-signature or unlock application), represented on the server-side by a DIGIPASS Application BLOB for each application each token.



User-Token Relationship for OneSpan Multi-Device

User's information is stored in the token license and the token batch. The token license is associated with the token instance for each device which contains the application name, authorization mode and its blob.



DPX Import Flow for OneSpan Multi-Device License

This import process includes extracting the DIGIPASS data and stores the DPX file specific information as well as details about the individual token instances in the DPX file. The following steps describe the DPX import flow.

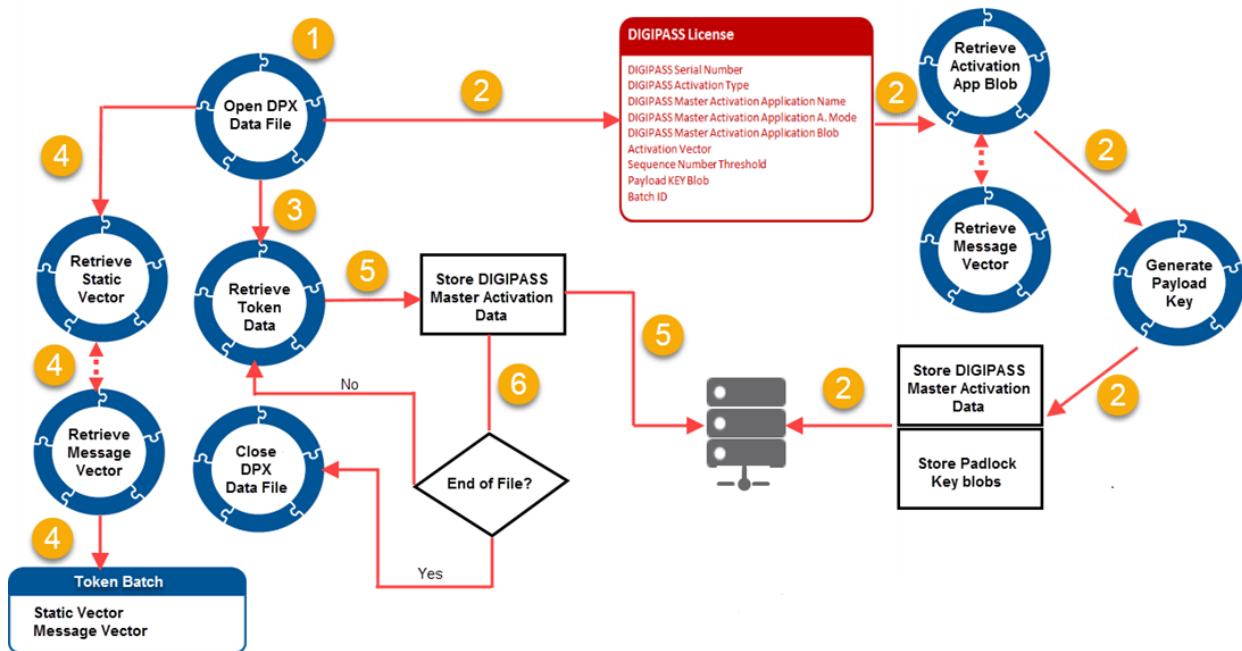
1. **Initialize DIGIPASS** - Open the DPX file using the DPX file name and the software transport key.

i **Notes:**

- The transport key is a 32-digits key encrypted with sensitive data in the DPX file through a 3DES mechanism.
- It is important to store both the DPX files and the transport key in a safe and secure location. OneSpan does not retain these files once their delivery has been made, and the administrator has to make sure that they are properly stored and accessible.

2. **Retrieve the DIGIPASS License** - Open the DFX file, extract the DIGIPASS license and its data such as DIGIPASS serial number, type, application names, authentication mode and its blob for the applications of a software DIGIPASS token.

3. **Store the DIGIPASS Data** - The DIGIPASS license data that are obtained in the previous step will be written into the database. The database will then be stored in the backend server as a token object.
4. **Retrieve the DIGIPASS static vector and message vector** - Retrieve them from the opened DPX file and store them in the token batch.
5. **Retrieve the DIGIPASS payload key blob** - Retrieve them from the DIGIPASS license stored in the database in step 3 and save them as the token's attributes.
6. **Close the DPX file.**



Notes:

- The static vector is a data string common for the DIGIPASS serial number in the DPX file that contains parameter settings. It will be used to generate activation data necessary for the activation process of a software DIGIPASS token.
- The message is a data string common for the DIGIPASS serial number in the DPX file that contains the configuration settings for the messages. VACMAN Controller will generate it for the activation process and the optional secure channel process.

Token Activation for OneSpan Multi-Device

AccessMatrix UAS supports both online and offline activation for OneSpan Cronto (Multi-Device) tokens. Both activations require the use of mobile token application generated by Digipass for Mobile version 4.6.2 and later. The hardware token DP760 is only supported for offline activation since it does not have any internet connectivity.

Standard Single Device Licensing Model

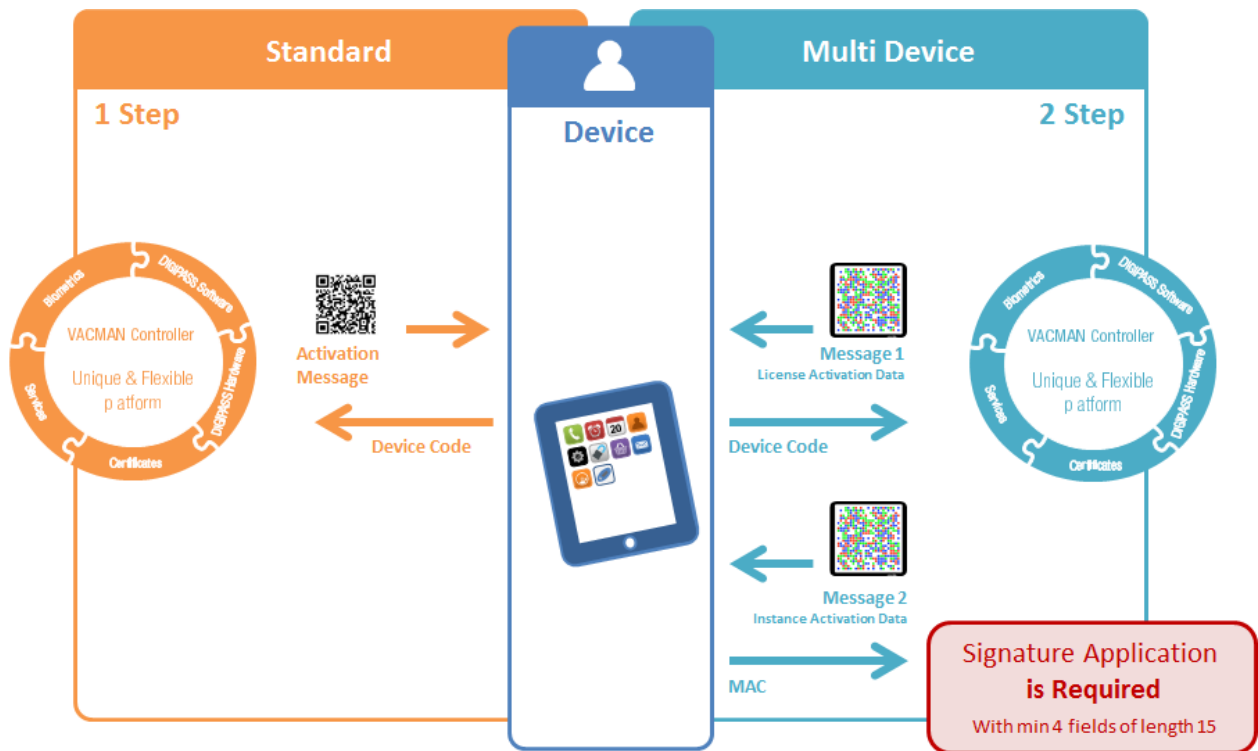
Before you can work with the DIGIPASS SDK, you need to activate it. To enable the DIGIPASS SDK, provide the activation data to the DIGIPASS SDK binary. The activation data includes the parameter settings, serial number, and the secret of a DIGIPASS token. During the DIGIPASS lifecycle, you may want to re-use the DIGIPASS serial number. For example, when re-installing the DIGIPASS to a new host platform (like a new mobile phone) or when you lose a DIGIPASS protection password. During the normal activation process, the event-based DIGIPASS uses an initial event counter set to 0. When the DIGIPASS activated that are used to validate responses, the DIGIPASS counters start incrementing at the server. By re-activating the same DIGIPASS on a new platform, the DIGIPASS counters are set to 0, while on the server, the counters have different values. By re-activating the same DIGIPASS on the same platform, the counters remain unchanged.

Multi-Device Licensing Model

Activate DIGIPASS SDK as a multi-device model includes providing the activation data to the DIGIPASS SDK binary which consists of the parameter settings, the serial number, the sequence number and the secret of a DIGIPASS token.

Contrary to the activation of the DIGIPASS in the single device licensing model, the activation data is provided in two steps to the DIGIPASS SDK: first, the license is activated. Then, the instance will be activated. Each phase consists of giving the client-side with the DIGIPASS SDK and the server-side with an activation message generated by VACMAN Controller.

Standard Single-Device vs Multi-Device Activation Flow Diagram



3. OneSpan Multi-Device Token Configuration

This section discusses the configurations required for setting up OneSpan Multi-Device token. Ensure that "UAS FEATURE" is enabled and the following vendors (depending on requirement):

"VASCO_MULTI_DEVICE, VASCO_MULTI_DEVICE_FM_ERACOM, VASCO_MULTI_DEVICE_FM_SAFENET, VASCO_MULTI_DEVICE_FM_THALES" are added in **Token Vendors** attribute of AccessMatrix server license.

Token Libraries

Before OneSpan Multi-Device tokens can be used with UAS server, there are library files required to be copied to AccessMatrix server folders:

1. Ensure that OneSpan Authentication Suite Server SDK (A3S) has been installed. Next, retrieve the following files from the A3S installation directory and copy to them to the appropriate AccessMatrix server folder. For example:

 Important Note: A3S upgrades may not be backward compatible. Before attempting an upgrade, ensure that you read AccessMatrix Product Upgrade Guide - Breaking Changes in OneSpan Authentication Suite Server SDK Upgrade .	
Windows	- o aal2sdk.dll - From <InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Bin\, copy to <UAS_DIR>\tomcat\bin\. - o aal2wrap.jar - From <InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Wrappers\Java\, copy to <UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib\.
Linux	- o libaal2sdk.so - From <InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Bin\, copy to the directory specified in the machine's java.library.path. - o aal2wrap.jar - From <InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Wrappers\Java\, copy to <UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib\.

2. Extract the Mobile Authenticator Studio **MAS_4.28.4_TwoStep.zip** file (use MAS version 4.28.4 and above). Then, retrieve the following files and copy them to the proper AM5 folder. For example:
 - o **DSAPP_Server.jar** - From <ExtractedDirectory>\MAS_4.28.4\Utilities\DSAPP Server SDK\Java\Bin\, copy to <UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib\.
 - o **ImageGeneratorSDK.jar** - From <ExtractedDirectory>\MAS_4.28.4\Utilities\Image Generator SDK\Java\Bin\, copy to <UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib\.
 - o **SecureMessagingSDKServer.jar** - From <ExtractedDirectory>\MAS_4.28.4\Utilities\Secure Messaging SDK Server\Java\Bin\, copy to <UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib\.
-

-
- **UtilitiesSDK.jar** - From <ExtractedDirectory>\MAS_4.28.4\Utilities\Secure Messaging SDK Server\Java\Bin\, copy to <UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib\.

**Notes:**

- If JAVA version is 32-bit, ensure that all the libraries used are in a 32-bit version. The same applies to the 64-bit system.
- UAS server can be started once all the libraries are being placed correctly. Check **am.log** for any errors during server startup. If any library is not compatible, the VASCO_MULTI_DEVICE, VASCO_MULTI_DEVICE_FM_ERACOM, VASCO_MULTI_DEVICE_FM_SAFENET, or VASCO_MULTI_DEVICE_FM_THALES token vendor handlers will not be registered/initialized successfully and, hence, cannot be used.

Token Policy

The token admin is required to configure token policy of OneSpan Multi-Device tokens. Token policy configuration can either be for HSM or non-HSM depending on the token vendor. For non-HSM configuration, a predefined OneSpan token policy, "DefaultVascoKernelParameters", is displayed if the required libraries are already added in AccessMatrix UAS. The license for this token is already enabled. To access this page, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Then, click and edit the "DefaultVascoKernelParameters" token policy.

A. Configure Secure Channel Application ID Keyword:

This step can be skipped if the Secure Channel application was not ordered. If Secure Channel was ordered, specify its Application ID/Name in the token policy under the **Secure Channel Application ID Keyword(s)** attribute. The Secure Channel Application ID/Name is the one specified in OneSpan's parameter sheet during the token order, e.g. "APP 4 SC". It allows the server to identify the Secure Channel Application.

B. Enable Score-Based Response feature:

If the score-based response feature is required, then there is a need to enable it in Token Policy as well. Under the **Digipass for APPS/Mobile Specific** tab, set the **Score-Based Response** attribute to "true". Skip this step if the feature is not required.

C. Online Activation Protocol (ECDH/SRP):

For online activation, the default selected protocol in AccessMatrix server is ECDH. If SRP is preferred, select the **Digipass for APPS/Mobile Specific** tab and modify the **Secured Data Exchange Protocol** attribute value to "SRP".

**Notes:**

- The OneSpan attributes of the policy will only be displayed if the OneSpan libraries are already placed in AccessMatrix server.
- For non-HSM configuration:
 - Configure the "VASCO Kernel Parameters" token policy. Refer to [UAS OneSpan Token Implementation Guide - UAS OneSpan Token Configuration](#) for details.
- For HSM configuration:
 - Configure the "VASCO Kernel Parameter (Thales SEE Enabled)" token policy. Refer to [UAS Entrust nShield Connect HSM Deployment Guide](#) for details.
 - Configure the "VASCO Kernel Parameters (SafeNet HSM FM Enabled)" token policy. Refer to [UAS Safenet ProtectServer HSM Deployment Guide](#) for details.

Token Batch

The administrator is required to create a token batch for physical grouping of OneSpan Multi-Device tokens. Creation can be done either in Admin Console or during token import using Token Import Utility (TIU). Ensure that the correct token policy is configured to specify the policy that will be used by the imported tokens. The default token policy for non-HSM configuration is "DefaultVascoKernelParameters".

Token Group

The administrator is required to create a token group for logical grouping of OneSpan token devices. Creation can be done either in the Admin Console or during token import using TIU. Ensure that the correct Token Vendor (e.g. "VASCO MULTI_DEVICE") is configured to group similar tokens.

Token Import

Secure the "VASCO DIGIPASS for Mobile" or "VASCO DIGIPASS for Apps" DPX file and associated transport key of the tokens, and log in to TIU to import the tokens.

1. Specify the VASCO DIGIPASS DPX file.
2. Select the token batch with the correct token policy.
3. Select the token group with the correct token vendor.
4. Specify the transport key value.
5. Click **Import Token**.

Once the token import is successful, the tokens can now be used for any authentication or authorization purposes.

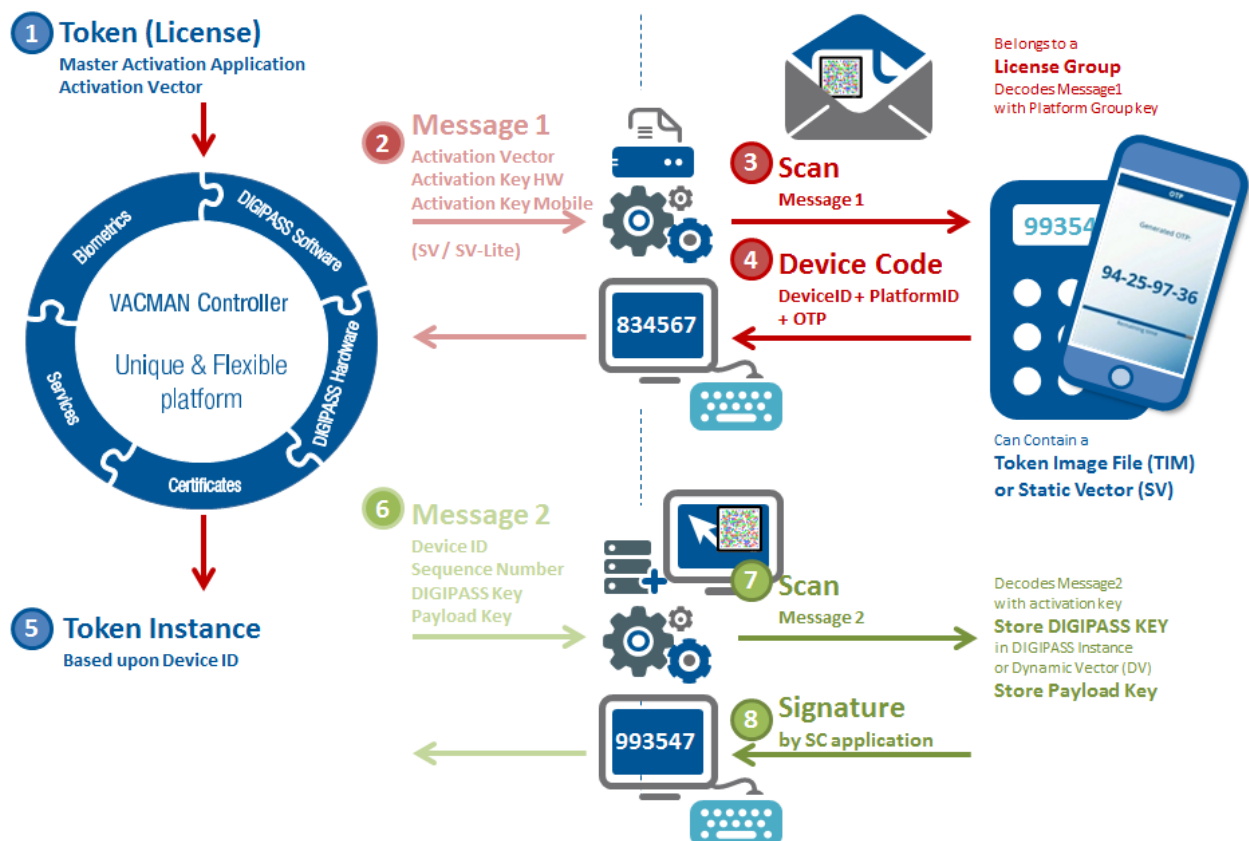
4. UAS OneSpan Multi-Device Implementation Using API

The following sections contains the UAS OneSpan multi-device implementation API use cases and their sample codes:

- [Multi-Device Token Activation](#)
- [2FA Login with OneSpan OTP](#)
- [Standard Verify OTP Only](#)
- [OneSpan Challenge-Response 2FA Login](#)
- [Other Challenge-Response Sample](#)
- [Standard Digital Signature Verification](#)
- [Secure Channel Verification](#)
- [Push Notification Feature](#)

4.1. Multi-Device Token Activation

The activation flow is different from the normal OneSpan Single Device Licensing token (which is our existing non-cronto supported tokens). It requires a two steps activation to successfully activate a Digipass either for hardware or mobile token. The following diagram shows the flow for 2-Steps activation process.



The purpose of the post-provisioning activation flow is for a customer to load a DIGIPASS instance into a DIGIPASS device. This flow is executed every time a new DIGIPASS instance needs to be loaded into the DIGIPASS device. The post-provisioning activation is also named secure activation.

The post-provisioning activation involves two messages generated by the customer. Two separated messages are used to guarantee that only the genuine end-user, and not an adversary who has intercepted one of the messages, can perform the activation.

Both activation messages should be delivered to the end-user via authentic channels. For instance, the first activation message (activation message 1) should be delivered via a secure letter or e-mail. The second activation message (activation message 2) should be delivered via the online banking application.

Activation message 1 can be used several times to activate multiple DIGIPASS instances (of a particular DIGIPASS license) on multiple DIGIPASS devices or only once if a challenge is used. On the other hand, activation message 2 can be used to activate one DIGIPASS instance only.

Refer to the illustration above for the multi-device software activation flow.

1. The post-provisioning activation process involves a particular DIGIPASS application named the Master Activation Application. It contains a master activation key different for each DIGIPASS Serial Number license. The VACMAN Controller must be provided with this Master Activation Application and the Activation Vector. The Activation Vector contains encrypted activation data that will be used to generate the Activation1Message for a particular DIGIPASS license.
 2. The VACMAN Controller generates "Message 1". Message 1 contains the Activation Vector. Optionally a Static Vector or a Static Vector Lite - The Static Vector Lite can update the response length and response checksum of a pre-stored static vector. This message is then rendered to a CrontoSign Image.
 3. The CrontoSign Image is delivered via letter to the end-user. This CrontoSign Image can then be scanned by an available DIGIPASS that belongs to the same license group as the license that was used for generating message1.
 4. The DIGIPASS creates a Device Code using the Static Vector, Message 1 and a platform fingerprint. This Device code must be delivered to the VACMAN Controller.
The second activation message (activation message 2) should be delivered via the online banking application.
 5. For each activation of a new DIGIPASS instance, new DIGIPASS applications Blobs are generated by the VACMAN Controller. A sequence number will be incremented for each new DIGIPASS instance issued from the same license. The number of instances which can be issued from a license will be limited to a pre-defined threshold between 1 and 99. The different DIGIPASS instances of one user account will have the same serial number but a different sequence number. The keys of the DIGIPASS instance applications will be different for each instance.
 6. Message 2 is created holding the DIGIPASS key and Payload Key. This message is then rendered to a CrontoSign image. Different DIGIPASS instances activated from one DIGIPASS serial number license must share the same Payload Key. After the activation, the Payload Key will protect the request messages and deactivation messages which are used to exchange between the server and the client devices. The messages have been activated using a DIGIPASS license (for a specific user account).
-

-
7. The CrontoSign Image is delivered via an online application. The CrontoSign Image can only be scanned by a DIGIPASS that consumed the previous message 1 and holds the proper Device Code.
 8. Generate a signature of Message 2 using the DIGIPASS Secure Message instance application and deliver it to the VACMAN Controller.

Sequence Diagram

Offline Activation Sequence Diagram

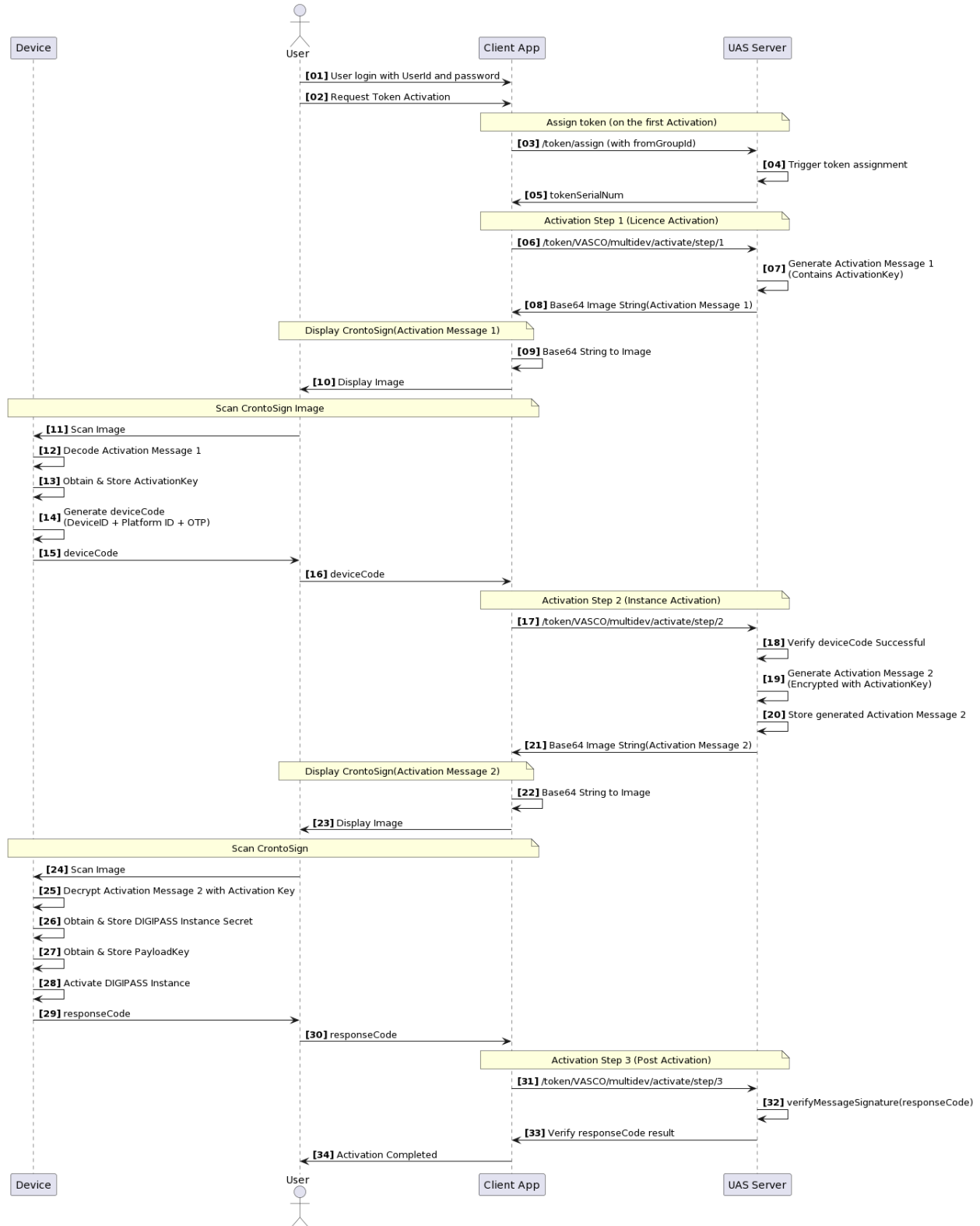
This section covers the two steps activation use case of OneSpan Multi-Device through REST Token Service API.

- The two-step activation is completed with 3 API calls, namely:
 1. `/token/VASCO/multidev/activate/step/1` - To generate Activation Message 1 (first CrontoSign) for License Activation
 2. `/token/VASCO/multidev/activate/step/2` - To verify the `deviceCode` generated after scanning Activation Message 1 (first CrontoSign); and if successful, generate Activation Message 2 (second CrontoSign) for Instance Activation
 3. `/token/VASCO/multidev/activate/step/3` - (Post Activation) To verify the `responseCode` generated after scanning Activation Message 2 (second CrontoSign); and if successful (i.e. no errors occur), the Activation is deemed successful and completed

For the complete sample code of all the above API Calls, refer to the [AccessMatrix REST API Specification](#) under the "Token Service - VASCO Multi Device Management/Verification Functions" section.

The sequence diagram below shows the basic flow of the API calls.

Multi-Device Offline Token Activation Sequence Diagram (Secure Channel Enabled)



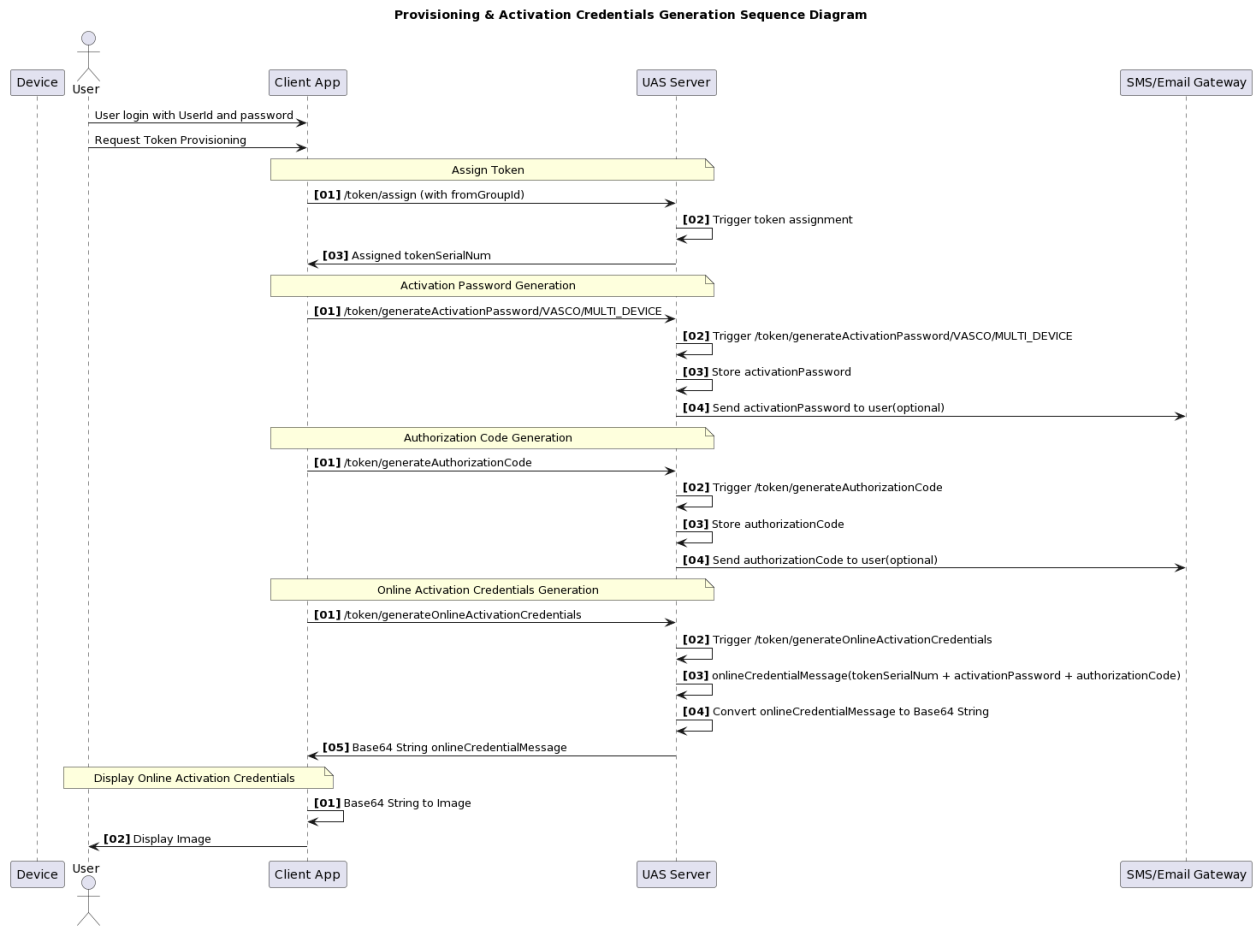
Online Activation (ECDH) Sequence Diagram

This section covers the online activation (via the Elliptic curve Diffie–Hellman(ECDH) protocol) use case of OneSpan Multi-Device through REST Token Service API.

-
- To use ECDH for online activation, make sure "ECDH" is selected as the **Secured Data Exchange Protocol** in the token policy. Refer to [Token Policy](#) under "C. Online Activation Protocol (ECDH/SRP)".
 - Online activation requires the client application to create and deploy a servlet/Webservice to handle the HTTP requests from the DP4Mobile APP.
 - The client application needs to prepare a servlet/Webservice for the mobile token app to call/invoke all the following 3 APIs for activation:
 1. `/token/VASCO/multidev/activate/step/1` - To generate Activation Message 1 (first CrontoSign) for License Activation.
 2. `/token/VASCO/multidev/activate/step/2` - To verify the `deviceCode` generated after Activation Message 1 is processed by the APP; and if successful, generate Activation Message 2 (second CrontoSign) for Instance Activation.
 3. `/token/VASCO/multidev/activate/step/3` - (Post Activation) To verify the `responseCode` generated after Activation Message 2 is processed by the APP; and if successful (no errors occur), Activation is deemed successful and completed.
 - For the complete sample code for all the above API Calls, refer to the [AccessMatrix REST API Specification](#) under the "Token Service - VASCO Multi-Device Management/Verification Functions" section.
 - To increase security during the delivery process for online activation, the activation data is protected by an encryption protocol based on the activation password, i.e. a shared secret between the server and the end-user.
 - The encryption protocol uses an encryption key based on a Diffie-Hellman shared secret which the server and the application derive from their private keys and the public key of the other party. Key pairs are generated according to an ECDH mechanism based on a NIST P-256 curve. This encryption protocol is used to encrypt the activation data during the delivery process between user and server.

The sequence diagram below shows the basic flow of the API calls.

This diagram shows the token provisioning and activation credentials generation flows. Activation credentials generation includes the activation password and authorization code generation. Activation password and authorization code can be generated and sent to users through different channels. There is also an option to embed activation password and authorization code inside the online activation credentials so that users can skip manual input for the activation password and authorization code.



This diagram shows how mobile app on users device initiate the online activation process once they scan the online activation credentials image and input activation password & authorization code if required. This diagram also shows how Diffie-Hellman Advanced Encryption is used in the online activation process.

Multi-Device Token Activation Sequence Diagram



Online Activation (SRP) Sequence Diagram

This section covers the online activation(via Secure Remote Password(SRP) protocol) use case of OneSpan Multi-Device through REST Token Service API.

- To use SRP for online activation, make sure SRP is selected as the Secured Data Exchange Protocol in the token policy. Refer to [Token Policy](#) under "C. Online Activation Protocol (ECDH/SRP)".

-
- Online activation requires the client application to create and deploy a servlet/Webservice to handle the HTTP requests from the DP4Mobile APP.
 - The client application needs to prepare a servlet/Webservice for the mobile token app to call/invoke all the following 4 APIs for activation:
 1. `/token/VASCO/multidev/SRP/serverEphemeralKey/generate` - To generate the server ephemeral public and private keys to be used subsequently to generate sessionKey for Activation Message 1 encryption. The sessionKey generation and Activation Message 1 encryption is done when `/token/VASCO/multidev/activate/step/1` is invoked.
 2. `/token/VASCO/multidev/activate/step/1` - To generate Activation Message 1 (first CrontoSign) for License Activation. In this context, the Activation Message 1 is encrypted by the sessionKey via the SRP protocol.
 3. `/token/VASCO/multidev/activate/step/2` - To verify deviceCode generated after Activation Message 1 is processed by the APP; and if successful, generate Activation Message 2 (second CrontoSign) for Instance Activation.
 4. `/token/VASCO/multidev/activate/step/3` - (Post Activation) To verify the responseCode generated after Activation Message 2 is processed by the APP; and if successful(no errors occur), the activation is deemed successful and completed.
 - For the complete sample code of all the above API Calls, refer to the [AccessMatrix REST API Specification](#) under the "Token Service - VASCO Multi-Device Management/Verification Functions" section.
 - To increase security during the delivery process for online activation, the activation data is protected by an encryption protocol based on the user password(SRP), i.e. a shared secret between the server and the end-user.
 - The SRP protocol creates a 256-bit AES session key shared between the two parties in a manner similar to Diffie–Hellman key exchange based on the client-side having the user password and the server-side having a cryptographic verifier derived from the password. The shared public key is derived from two random numbers, one generated by the client, and the other generated by the server, which is unique to the activation attempt. This encryption protocol is used to encrypt the activation data during the delivery process between user and server.
 - With this protocol (SRP), the secret shared between the server and the client – the SRP user password – is not transmitted through the network. The SRP user password must be generated by the server and is bound to a unique identifier, i.e. the user identity (usually is the token serial number in this case). The SRP user password must be securely transmitted to the user via a
-

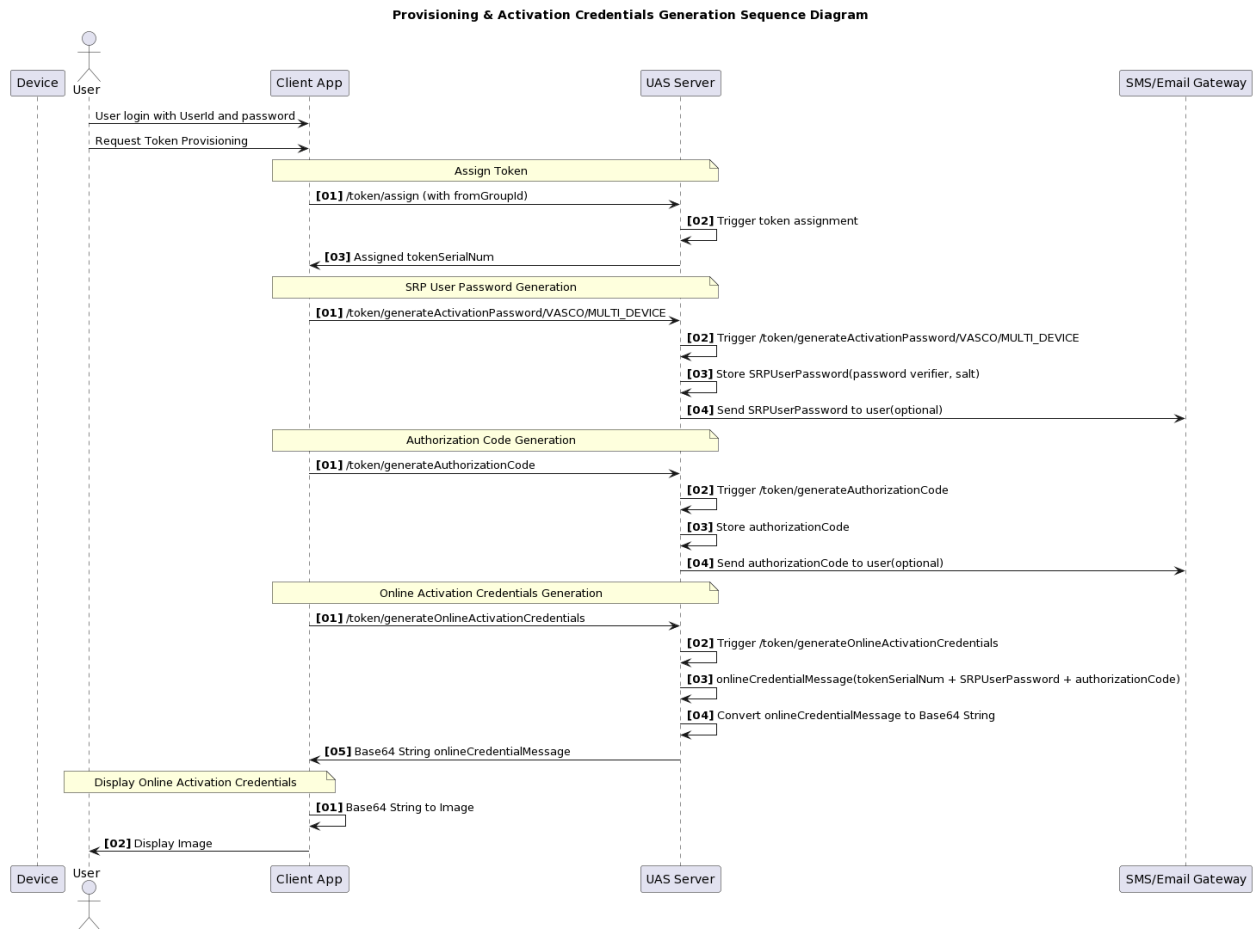
separate channel outside the network. The user will then enter their SRP user password in the mobile client application OR scan the QRCode containing such credentials using the mobile application.

The sequence diagram below shows the basic flow of the API calls.

This diagram shows the token provisioning and activation credentials generation flows. Activation credentials generation includes the SRP user password and authorization code generation. The SRP user password and authorization code can be generated and sent to users through different channels. There is also an option to embed SRP user password and authorization code inside the online activation credentials so that users can skip manual input for the SRP user password and authorization code.

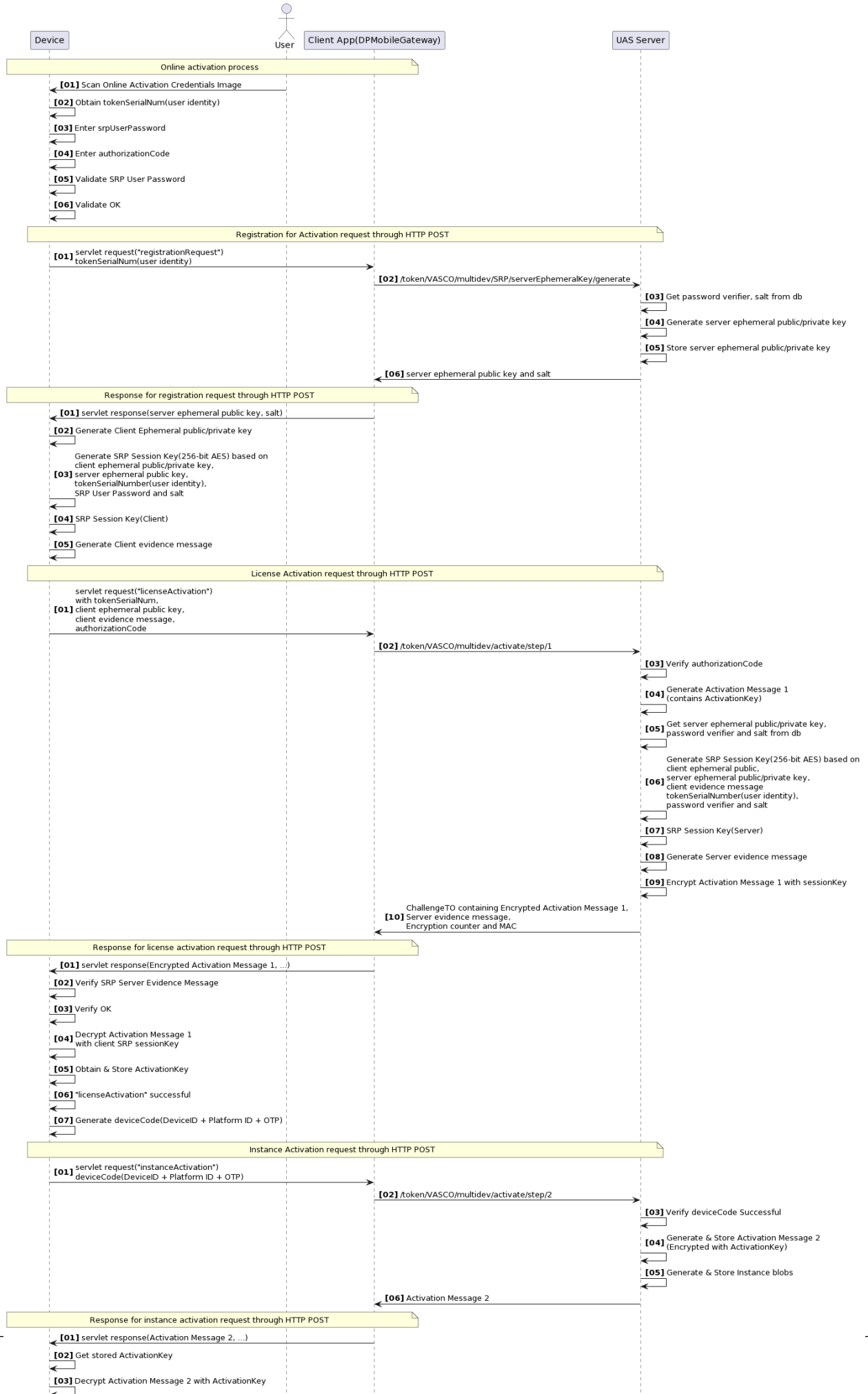
The following APIs are required for Activation Credentials Generation:

1. `/token/generateActivationPassword/VASCO/MULTI_DEVICE` - To generate the SRP user password, password verifier and salt. This API is mandatory for SRP to work. SRP user password can then be delivered to the user through a separate, secure channel(SMS/Email) or it can be embedded into the Activation Credentials QRCode/CrontoSign. Password verifier and salt will be stored in the server database.
2. `/token/generateAuthorizationCode` - To generate authorization code. This API is optional. The authorization code is used to verify if the activation is being requested by a valid user. The authorization code can be delivered to the user through a separate, secure channel (SMS/Email) or it can be embedded into the Activation Credentials QRCode/CrontoSign.
3. `/token/generateOnlineActivationCredentials` - To generate the Activation Credentials QRCode/CrontoSign. This QRCode/CrontoSign will contain the token serial number, authorization code (optional, otherwise the user can manually input), SRP user password (optional, otherwise the user can manually input) and user identity (optional).



This diagram shows how mobile app on users device initiate the online activation process once they scan the online activation credentials image and input SRP user password & authorization code if required. This diagram also shows how SRP Protocol is used in the online activation process.

Multi-Device Token Activation Sequence Diagram



REST Sample Code

Online Activation Credentials Sample Code

This section provides the sample code for token assignment, activation password & authorization code generation, token status change and online activation credentials generation.

1. `/token/assign` (with `fromGroupId`) assigns an available token the user automatically.
2. `/token/generateActivationPassword/VASCO/MULTI_DEVICE` generates the Activation Password. Activation password is used to encrypt the Activation Data on the later part, so it is a shared secret between user and server. This Activation Password can be sent to the user through a different channel or to be embedded in the Online Activation Credentials (CrontoSign image).
3. `/token/generateAuthorizationCode` generates the Authorization Code. Authorization Code is used to verify the user during activation whereby the server will match the authorization code stored in the server with the one user inputs. If authorization code mismatched, the activation process will be terminated immediately on the server-side. This Authorization Code can be sent to the user through a different channel or to be embedded in the Online Activation Credentials (CrontoSign image).
4. `/token/changeStatus` is called to change the token status as required depending on the provisioning/activation process.
5. `/token/generateOnlineActivationCredentials` generates a CrontoSign image which contains the token serial number. This CrontoSign generated is meant for online activation, it can be sent to the user through a different channel or displayed directly to the user on the client's application. Users will scan this CrontoSign image to start the online activation process. This API has the option to embed the Activation Password and Authorization Code generated in Step 3 & 4. If the Activation Password and Authorization Code are embedded, users can just scan the Online Activation Credentials without the need to manually input the Activation Password and Authorization Code.

Token Assignment	
URL: <code>/token/assign</code>	
Request	
	JSON

Token Assignment	
	<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "fromGroupId": "TOKEN_GROUP_1", "vendor": "VASCO_MULTI_DEVICE", "userId": "testuser" }</pre>
Response	
	JSON
	<pre>{ "result": { "challenge": { "params": { "assignedTokenSerialNumbers": ["0123456"] } } } }</pre>
Authorization Code Generation	
URL: /token/generateAuthorizationCode	
Request	
	JSON
	<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "eventNotificationPolicyId": "DPMobileEmailAuthorizationCodePolicy" }</pre>
Response	
	JSON
	<pre>{ "result": { "isGenerated": true, "authorizationCode": "9678543021" } }</pre>
Activation Password Generation	
URL: /token/generateActivationPassword/VASCO/MULTI_DEVICE	

Token Assignment

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "eventNotificationPolicyId":
  "DPMobileSMSActivationCodePolicy",
  "midletLink": "http://host:port/download"
}
```

Response

JSON

```
{
  "result": {
    "isGenerated": true
  }
}
```

Token Status Change

URL: /token/changeStatus

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "status": "PreActivation",
}
```

Response

JSON

```
{
  "result": {
    "challenge": {
      "params": {
        "tokenSerNum": "0123456",
        "tokenStatus": "PreActivation",
        "vendorId": "VASCO_MULTI_DEVICE",
        "tokenUUID": "XVkp0Mj6LH",
        "oldTokenStatus": "Activated"
      }
    }
  }
}
```

Token Assignment
<pre> } } </pre>
Online Activation Credentials Generation
URL: /token/generateOnlineActivationCredentials
Request JSON <pre> { "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "userIdentifier": "testuser", "withActivationPassword": true, "withAuthorizationCode": true, "params": { "instanceFriendlyName": "MyToken" } } </pre>
Response JSON <pre> { "result": { "params": { "imageBase64": "0002nEZMqgYDmlfsYwRvzi_QSV_iOzASliem....TvNGKd2mGTyhvBRkA4rVwQidCD4Noe7FHd6 MrvKZ-__rk" } } } </pre>

Online Activation Credentials Sample Code (SRP)

This section provides the sample code for token assignment, SRP user password & authorization code generation, token status change and online activation credentials generation.

1. /token/assign (with fromGroupId) assigns an available token to the user automatically.
2. /token/generateActivationPassword/VASCO/MULTI_DEVICE generates the SRP User Password. SRP User Password is used to encrypt the Activation Data on the later part, so it is a shared secret between user and server. This SRP User Password can be sent to the user through a

different channel or to be embedded in the Online Activation Credentials (QRCode/CrontoSign image).

3. `/token/generateAuthorizationCode` generates the Authorization Code. Authorization Code is used to verify the user during activation whereby the server will match the authorization code stored in the server with the one user inputs. If authorization code mismatched, the activation process will be terminated immediately on the server-side. This Authorization Code can be sent to the user through a different channel or to be embedded in the Online Activation Credentials (CrontoSign image).
4. `/token/changeStatus` is called to change the token status as required depending on the provisioning/activation process.
5. `/token/generateOnlineActivationCredentials` generates a CrontoSign image which contains the token serial number. This CrontoSign generated is meant for online activation, it can be sent to the user through a different channel or displayed directly to the user on the client's application. Users will scan this CrontoSign image to start the online activation process. This API has the option to embed the Activation Password and Authorization Code generated in Step 3 & 4. If the Activation Password and Authorization Code are embedded, users can just scan the Online Activation Credentials without the need to manually input the Activation Password and Authorization Code.

Token Assignment	
URL: <code>/token/assign</code>	
Request	
JSON	<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "fromGroupId": "TOKEN_GROUP_1", "vendor": "VASCO_MULTI_DEVICE", "userId": "testuser" }</pre>
Response	
JSON	<pre>{ "result": { "challenge": { "params": {</pre>

Token Assignment

```
{
  "assignedTokenSerialNumbers": ["0123456"]
}
```

Authorization Code Generation

URL: /token/generateAuthorizationCode

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "eventNotificationPolicyId":
    "DPMobileEmailAuthorizationCodePolicy"
}
```

Response

JSON

```
{
  "result": {
    "isGenerated": true,
    "authorizationCode": "9678543021"
  }
}
```

SRP User Password Generation

URL: /token/generateActivationPassword/VASCO/MULTI_DEVICE

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "eventNotificationPolicyId":
    "DPMobileSMSActivationCodePolicy",
  "midletLink": ""
}
```

Token Assignment	
Response	
JSON	
<pre>{ "result": { "isGenerated": true } }</pre>	
Token Status Change	
URL: /token/changeStatus	
Request	
JSON	
<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "status": "PreActivation", }</pre>	
Response	
JSON	
<pre>{ "result": { "challenge": { "params": { "tokenSerNum": "0123456", "tokenStatus": "PreActivation", "vendorId": "VASCO_MULTI_DEVICE", "tokenUUID": "XVkp0Mj6LH", "oldTokenStatus": "Activated" } } } }</pre>	
Online Activation Credentials Generation Sample Code	
URL: /token/generateOnlineActivationCredentials	
Request	
JSON	

Token Assignment	
<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "userIdentifier": "testuser", "withActivationPassword": true, "withAuthorizationCode": true, "params": { "instanceFriendlyName": "MyToken" } }</pre>	
Response	
JSON	
<pre>{ "result": { "params": { "imageBase64": "0002nEZMqgYDmlfsYwRvzi_QSV_iOzASliem....TvNGKd2mGTyhvBRkA4rVwQidCD4Noe7FHd6 MrvKZ-__rk" } } }</pre>	

Sample Servlet Code For License Activation (ECDH)

After the user enters the token serial number or scan the CrontoSign image (Online Activation Credentials) containing the token serial number and enter the Activation Password (received by email) on their mobile token app, the activation will start automatically. The mobile app will send a request to the client application, in this sample we are using a Java servlet. The mobile app will be making a "licenseActivation" HTTP request to the client application.

When client application receives the "licenseActivation" request, the /token/VASCO/multidev/activate/step/1 API should be called. This API generates Activation Message 1 and returned as base64 encoded image string (CrontoSign image). For online activation, the Activation Message 1 generated is encrypted and other information (LICENSE_ACTIVATION_MESSAGE_IV, ENCRYPTED_SERVER_PUBLIC_KEY, ENCRYPTED_NONCES and GENERATE_SESSION_KEY_IV) is returned. The Activation Message 1 is encrypted with the Activation Password that was generated and sent to the user by email. In this sample, the encrypted Activation Message 1 is called ENCRYPTED_LICENSE_ACTIVATION_MESSAGE.

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/1

URL: /token/VASCO/multidev/activate/step/1

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "isOnlineMode": "true",
  "params": {
    "authorizationCode": "9678543021",
    "publicKey": "ABCD12345....97EE0F",
    "initialVector": "AV013445BF64KC",
    "deviceIdentifier": "M0123456789"
  }
}
```

Response

JSON

```
{
  "result": {
    "params": {
      "retCode": "0",
      "encryptedLicenseActivationMessage":
"HBkI0....Julktz",
      "licenseActivationMessageIV": "XUpI2owaR6O",
      "encryptedServerPublicKey": "3o_WN....FzrX",
      "encryptedNonces": "Okly....gYaJ",
      "generateSessionKeyIV": "x9yDBNz6f2Msna",
      "duplicatedDeviceUniqueId": "false"
    }
  }
}
```

The servlet should then construct the `EncryptedLicenseActivationMessageResponse` and put in into the servlet response

- if "duplicatedDeviceUniqueId" is "true", the servlet should throw an exception with a message like "An activated Digipass instance has been detected on the server for this device. Please contact your Administrator."
- else construct `EncryptedLicenseActivationMessageResponse` and return it to the app

Java

```
EncryptedLicenseActivationMessageResponse  
encryptedLicenseActivationMessageResponse = new
```


Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/1

```
EncryptedLicenseActivationMessageResponse (
    /**"encryptedLicenseActivationMessage":*/
    "HBkIO....Julktz",
    /**"licenseActivationMessageIV":*/ "XUpI2owaR6O",
    /**"encryptedServerPublicKey":*/ "3o_WN....FzrX",
    /**"encryptedNonces":*/ "Okly....gYaJ",
    /**"generateSessionKeyIV":*/ "x9yDBNz6f2Msna",
    );
```

DIGIPASS.xml Configuration To Generate Mobile App That Will Call The Servlet For "licenseActivation" Request

XML

```
<MultiDeviceActivation
id="multiDeviceOnlineImageActivation">

    <!-- Graphical data associated with the select
activation screen -->
    <MenuItem value="Scan credentials for online activation"
icon="activate_multi_device.png" class="menuItem"/>

    <!-- License activation configuration & associated
graphical data -->
    <LicenseActivation useActivationPassword="true"
checksumOnActivationPassword="false">

        <OnlineLicenseActivation
useRegistrationIdentifier="true" useAuthorizationCode="true"
checksumOnAuthorizationCode="false" inputType="image"
imageFormat="all" >
            <URL method="POST"
value="http://192.168.50.107:8080/dp4mobile-
cronto/DPMobileGateway">
                <PayloadParameter key="action"
value="licenseActivation" />
                <PayloadParameter
key="registrationIdentifier"
value="%_RegistrationIdentifier_" />
                <PayloadParameter key="authorizationCode"
value="%_AuthorizationCode_" />
                <PayloadParameter key="publicKey"
value="%_PublicKey_" />
                <PayloadParameter key="initialVector"
value="%_InitialVector_" />
                <PayloadParameter key="deviceIdentifier"
value="%_DeviceIdentifier_" />
            </URL>
        </OnlineLicenseActivation>
```

Sample Servlet Code For Registration Request (SRP)

After the user enters the token serial number or scan the QRCode/CrontoSign image (Online Activation Credentials) containing the token serial number and enter the SRP User Password (received by email, SMS or by scanning the one embedded in QRCode/CrontoSign image) on their mobile token app, the activation will start automatically. The mobile app sends a request to the client application - in this sample, the Java servlet is used. The mobile app will be making a "registrationRequest" HTTP request to the client application.

When the client application receives the "registrationRequest" request, the `/token/VASCO/multidev/SRP/serverEphemeralKey/generate` API should be called. This API generates server ephemeral public and private key pair and store them into the database. Server ephemeral key pairs are required in the latter part for the SRP sessionKey (server) generation, which plays an important role for Activation Message 1 encryption using SRP protocol.

`/token/VASCO/multidev/SRP/serverEphemeralKey/generate` will then return the server ephemeral public key and salt to the mobile app which will be used for SRP sessionKey (client) generation.

Online Registration Request Servlet Example - Trigger <code>/token/VASCO/multidev/SRP/serverEphemeralKey/generate</code>	
URL: <code>/token/VASCO/multidev/SRP/serverEphemeralKey/generate</code>	
Request	
JSON	<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "authorizationCode": "9678543021", }</pre>
Response	
JSON	<pre>{ "result": { "serverEphemeralPublicKey": "00012N4rHoRRWBUWXNZ1...MGLF_I3DUYhoXnqvLfM1E", "salt": "J1zAOX9nIW9FEs2ftUuiN6FPI4ebjQOJp" } }</pre>

Sample Servlet Code For License Activation (SRP)

Once the registration request is completed [see [Sample Servlet Code For Registration Request\(SRP\)](#)], the mobile app makes an HTTP request for "licenseActivation" to the client application.

When the client application receives the "licenseActivation" request, the

/token/VASCO/multidev/activate/step/1 API should be called. This API generates Activation Message 1. For online activation, the Activation Message 1 generated is encrypted and also other information

(SERVER_EVIDENCE_MESSAGE, ENCRYPTED_LICENSE_ACTIVATION_MESSAGE_MAC, and

ENCRYPTED_LICENSE_ACTIVATION_MESSAGE_ENCRYPTION_COUNTER) will be returned. The Activation

Message 1 is encrypted with the SRP session key generated based on the client ephemeral public key

(generated by mobile app), client evidence message (generated by mobile app), server ephemeral

public/private key (generated by the /token/VASCO/multidev/SRP/serverEphemeralKey/generate API),

token serial number, password verifier (generated by the

/token/generateActivationPassword/VASCO/MULTI_DEVICE API), and salt (generated by the

/token/generateActivationPassword/VASCO/MULTI_DEVICE API). In this sample, the encrypted Activation

Message 1 is called ENCRYPTED_LICENSE_ACTIVATION_MESSAGE.

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/1	
URL: /token/VASCO/multidev/activate/step/1	
Request	
JSON	
<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "isOnlineMode": "true", "params": { "authorizationCode": "9678543021", "clientEphemeralPublicKey": "ABCD12345....97EE0F", "clientEvidenceMessage": "AV013445BF64KC", "deviceIdentifier": "M0123456789" } }</pre>	
Response	
JSON	

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/1

```
{
  "result": {
    "params": {
      "retCode": "0",
      "encryptedLicenseActivationMessage":
"HBkI0....Julktz",
      "encryptedLicenseActivationMessageMAC": "XUpI2owaR6O",
      "serverEvidenceMessage": "3o_WN....FzrX",
      "encryptedLicenseActivationMessageEncryptionCounter":
"1C6D7944",
      "duplicatedDeviceUniqueId": "false"
    }
  }
}
```

The servlet should then construct the `EncryptedLicenseActivationMessageResponse` and put in into the servlet response

- if "duplicatedDeviceUniqueId" is "true", the servlet should throw an exception with a message like "An activated Digipass instance has been detected on the server for this device. Please contact your Administrator."
- else return all the values "encryptedLicenseActivationMessage", "encryptedLicenseActivationMessageMAC", "encryptedLicenseActivationMessageEncryptionCounter", "serverEvidenceMessage" to the app

Sample Servlet Code For Instance Activation

After the "licenseActivation" servlet request is processed successfully, the mobile app triggers the second request to the servlet. In this sample case, it will make an "instanceActivation" HTTP request to the client application servlet.

This step triggers /token/VASCO/multidev/activate/step/2, similar to Offline Activation; the difference is that the Activation Message 2 is generated and sent directly to the mobile app (user does not need to scan the Activation Message 2). This process is completely transparent to the user.

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/2

URL: /token/VASCO/multidev/activate/step/2

Request

JSON

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/2

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "deviceCode": "5BB298",
  "deviceFriendlyName": "MyToken"
}
```

Response

JSON

```
{
  "result": {
    "params": {
      "retCode": "0",
      "signedMessage":
"03F96D794431415130323031....3764D6C0557C843734C7B88EDF80"
    }
  }
}
```

If the "retCode" is "0", it is successful and return "signedMessage" to the app

DIGIPASS.xml Configuration To Generate Mobile App That Will Call The Servlet For "instanceActivation" Request

XML

```
<InstanceActivation>

  <OnlineInstanceActivation >
    <URL method="POST"
value="http://192.168.50.107:8080/dp4mobile-
cronto/DPMobileGateway">
      <PayloadParameter key="action"
value="instanceActivation" />
      <PayloadParameter key="serialNumber"
value="%_SerialNumber_" />
      <PayloadParameter key="deviceCode"
value="%_DeviceCode_" />
    </URL>
  </OnlineInstanceActivation>

  <View>
    <Labels>
      <Label id="MultiDeviceLicenseActivationSuccess"
value="You have preactivated your license. Please find below
your device code."/>
      <Label id="GetInstanceActivationWaitMessage"
value="Activating instance, please wait..."/>
    </Labels>
  </View>
</InstanceActivation>
```

DIGIPASS.xml Configuration To Generate Mobile App That Will Call The Servlet For "instanceActivation" Request

```
</Labels>
<Dynamics>
  <Dynamic id="DeviceCode"
class="resultGenerated"/>
</Dynamics>
</View>

</InstanceActivation>
```

Sample Servlet Code For Post Activation

After the "instanceActivation" servlet request is processed successfully, the mobile app triggers another request to the servlet. In this sample case, it will make the "postActivation" request to the client application servlet.

This step triggers /token/VASCO/multidev/activate/step/3, similar to Offline Activation; the difference is that this process is completely transparent to the user. This final step will complete the activation if successful.

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/3

URL: /token/assignee/list

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
}
```

Response

JSON

```
{
  "result": [
    {
      "id": "testuser",
      "UUID": "4314151303C843734C7B88EDF80",
      "name": "Test User"
    }
  ]
}
```

Online Activation Servlet Example - Trigger /token/VASCO/multidev/activate/step/3

```
]
}
```

To check if token is really assigned to the user

URL: /token/VASCO/multidev/activate/step/3

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "responseCode": "587298",
}
```

Response

JSON

```
{
  "result": {
    "params": {
      "retCode": "0",
    }
  }
}
```

If the "retCode" is "0", it is successful

DIGIPASS.xml Configuration To Generate Mobile App That Will Call The Servlet For "postActivation" Request

XML

```
<PostActivation cryptoAppIndex="4">
  <OnlinePostActivation destroyOnFailure="true"
displayMessageOnSuccess="false">
    <URL method="POST"
value="http://192.168.50.107:8080/dp4mobile-
cronto/DPMobileGateway">
      <PayloadParameter key="action"
value="postActivation" />
      <PayloadParameter key="serialNumber"
```

DIGIPASS.xml Configuration To Generate Mobile App That Will Call The Servlet For "postActivation" Request

```
value="%_SerialNumber_" />
    <PayloadParameter key="sequenceNumber"
value="%_SequenceNumber_" />
    <PayloadParameter key="otp" value="%_OTP_" />
    <PayloadParameter key="userIdentifier"
value="%_UserIdentifier_" />
    </URL>
</OnlinePostActivation>

    <View title="Post activation">
        <Labels>
            <Label id="OnlineServerActivationWaitMessage"
value="Activation confirmation, please wait..." />
        </Labels>
        <Errors>
            <Error id="ChallengeEmpty" value="The activation
response does not include a challenge." />
            <Error id="ChallengeTooShort" value="The minimal
length for the challenge is %_MinLength_" />
            <Error id="ChallengeTooLong" value="The maximal
length for the challenge is %_MaxLength_" />
        </Errors>
    </View>
</PostActivation>
```

Possible Parameters & Exceptions

Refer to "Token Service - VASCO Multi-Device Management/Verification Functions" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

Responses

Refer to "Token Service - VASCO Multi-Device Management/Verification Functions" in the [AccessMatrix REST API Specification](#) for all possible responses.

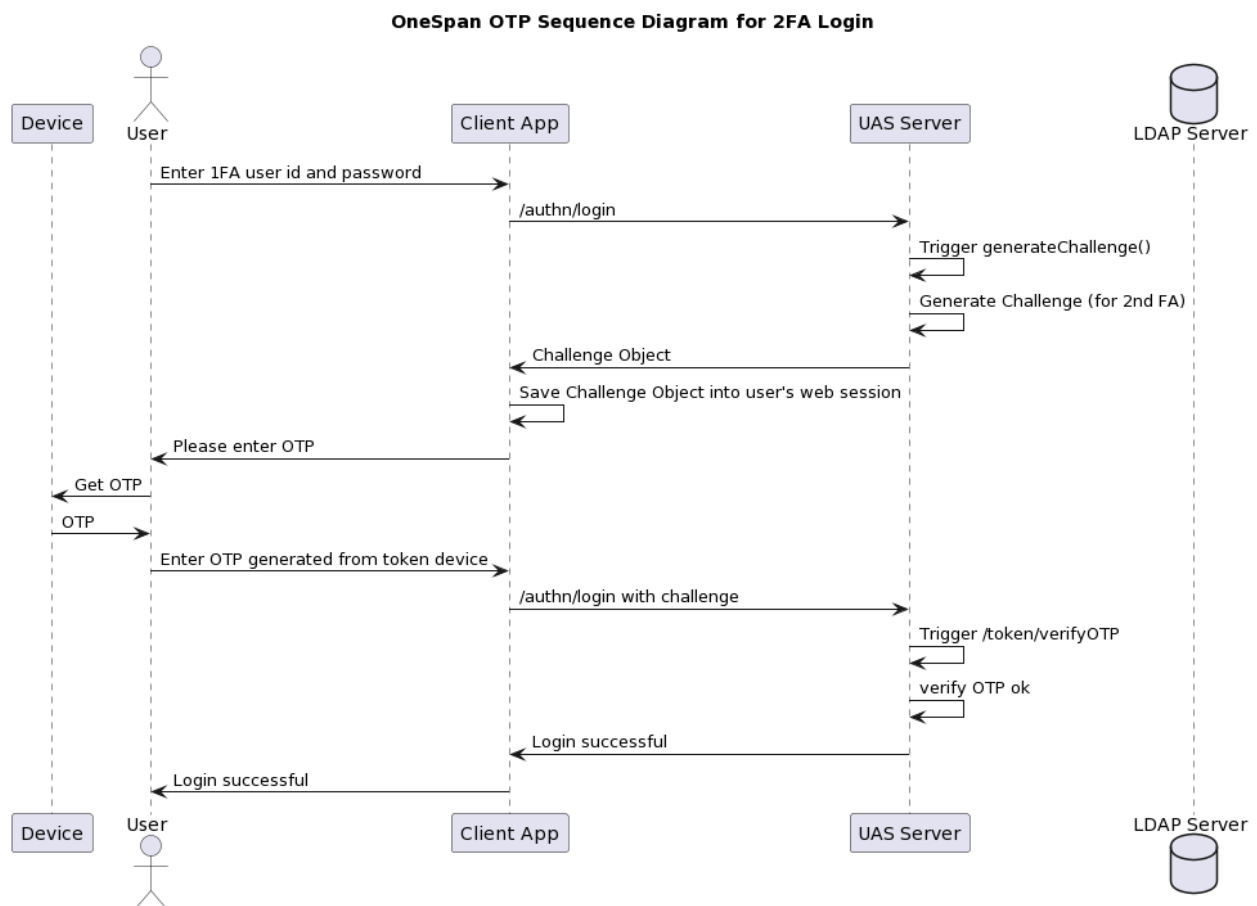
4.2. 2FA Login with OneSpan OTP

Sequence Diagram

This section covers the two-factor authentication use case of OneSpan Token OTP through the REST Service `/authn/login` API and also the REST Token Service `/token/verifyOTP` API.

The sequence diagram below shows the basic flow of the API calls for OneSpan (VASCO) Token OTP Login.

1. By calling the `/authn/login` API, UAS generates a challenge if the authentication realm passed in as a parameter is defined for VascoTokenLogin. The challenge is then used together with OTP from Token Device for 2nd Factor Login.
2. The user should pass the authentication realmId.
3. Invoke `/authn/login` with the challenge for 2nd Factor Login.
4. password is the OTP generated from the token device and challenge is from the 1st login call.



REST Sample Code

2FA Login API Calls Sample Code
URL (1FA with password): /authn/login
Request
JSON
<pre>{ "userId": "otpuser", "realmId": "VascoOtp2FA", "password": "mypassword1", }</pre>
Response
JSON
<pre>{ "result": { "sessionToken": "10001hUX9....0LtRMKes", "challenge": { "message": "Please enter password for next login module", "params": { } } } }</pre>
URL (2FA with token OTP): /authn/login
Request
JSON
<pre>{ "sessionToken": "10001yMEF....LebTxm2eM", "userId": "otpuser", "realmId": "VascoOtp2FA", "password": "244097", }</pre>

2FA Login API Calls Sample Code

Response

JSON

```
{
  "result": {
    "sessionToken": "10001hUX9....0LtRMKes",
    "authenticated": true,
  }
}
```

Possible Parameters & Exceptions

Refer to "Authentication Service" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

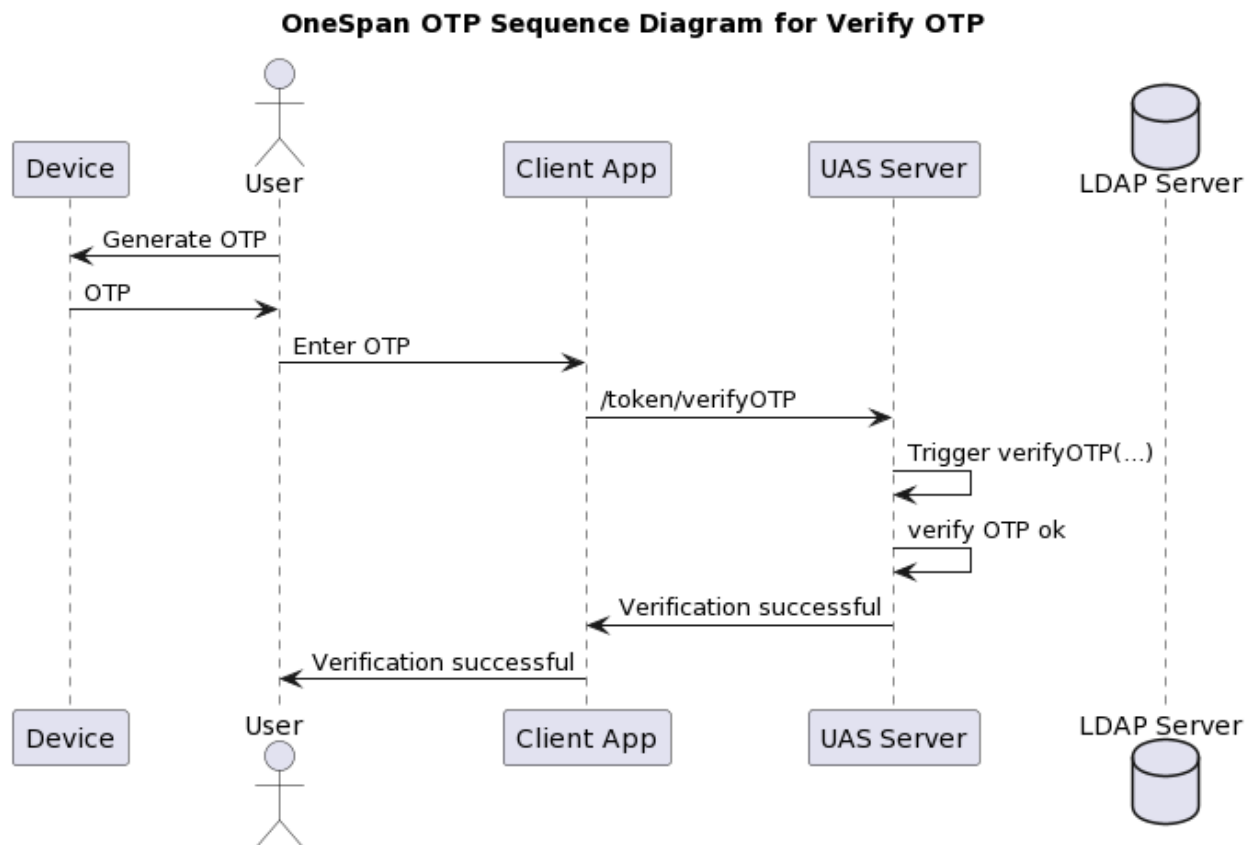
Responses

Refer to "Authentication Service" in the [AccessMatrix REST API Specification](#) for all possible responses.

4.3. Standard Verify OTP Only

Sequence Diagram

This section depicts the standard verifyOTP-only use case. It is used specifically to verify the user's token OTP without any login authentication.



REST Sample Code

VerifyOTP API Call Sample Code	
URL: /token/verifyOTP	
Request	
JSON	
<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456",</pre>	

VerifyOTP API Call Sample Code

```
{  
  "OTP": "980242"  
}
```

Response

JSON

```
{  
  "result": {  
    "params": {  
      "deviceType": "Android",  
      "deviceFriendlyName": "MyToken",  
      "instanceNumber": "1"  
    }  
  }  
}
```

Possible Parameters & Exceptions

Refer to "Token Service (/token/verifyOTP)" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

Responses

Refer to "Token Service (/token/verifyOTP)" in the [AccessMatrix REST API Specification](#) for all possible responses.

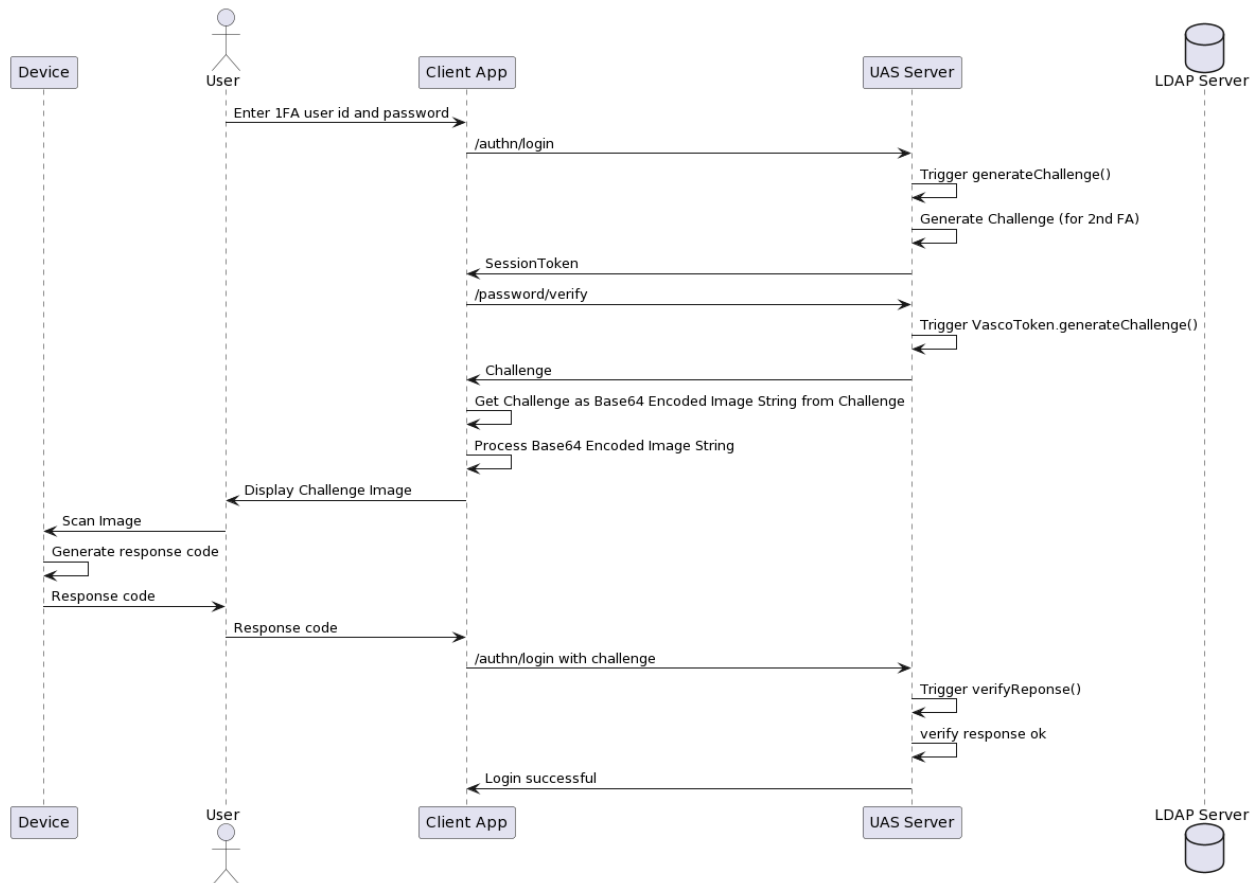
4.4. OneSpan Challenge-Response 2FA Login

Sequence Diagram

The sequence diagram below shows the basic flow of the API calls for OneSpan Challenge-Response 2FA Login.

1. By calling the `/authn/login` API, UAS generates the challenge if the authentication realm passed in as a parameter is configured for 2FA with `VascoTokenLogin`. This challenge is to prompt the user for the 2nd-factor Login.
 2. The user should pass the authentication `realmId` as one of the parameters.
 3. The user also needs to pass in `authMode` as one of the parameters in the map `params` to define the Challenge-Response "CR".
 4. The `sessionToken` returned from 1st login should be stored for use in 2nd login.
 5. `password/verify` is called with a blank password field to generate the challenge object.
 6. A challenge string and base64 encoded image string of the challenge will be returned together inside the challenge object.
 7. The client app should process and display the base64 encoded image string to the user.
 8. The user scans the image using their token device to generate a response.
 9. Invoke `/authn/login` with a challenge for 2nd-factor login.
 10. The `sessionToken` from the 1st login is used as the `sessionToken` for the 2nd login.
 11. `password` will be the response generated from the token device and `challenge` is from verifying the password.
 12. Authentication realm should be passed as `realmId`.
-

OneSpan Challenge-Response Sequence Diagram for 2FA Login



REST Sample Code

2FA Login with Challenge Response API Calls Sample Code	
URL (1FA with CR for 2FA): /authn/login	
Request	
JSON	
<pre>{ "userId": "aduser", "realmId": "Vasco2FA", "password": "myPassword1", "params": { "authMode": "CR" } }</pre>	
Response	
JSON	

2FA Login with Challenge Response API Calls Sample Code

```
{
  "result": {
    "sessionToken": "10001hUX9....0LtRMKes",
    "challenge": {
      "challengeToken":
"03F96D794431415130323031....3764D6C0557C843734C7B88EDF80"
      "params": {
        "imageBase64":
"0002nEZMqgYDmlfsYwRvzi_QSV_iOzASliem....TvNGKd2mGTyhvBRkA4rVwQidCD4Noe7FHd6
MrvKZ-__rk",
        "challenge":
"03F96D794431415130323031....3764D6C0557C843734C7B88EDF80"
      }
    }
  }
}
```

URL (2FA with response from token): /authn/login

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "userId": "aduser",
  "realmId": "Vasco2FA",
  "password": "654923",
  "challengeToken":
"03F96D794431415130323031....3764D6C0557C843734C7B88EDF80"
  "params": {
    "authMode": "CR"
  }
}
```

Response

JSON

```
{
  "result": {
    "sessionToken": "10001yMEF....LebTxm2eM",
    "authenticated": true
  }
}
```

Possible Parameters & Exceptions

Refer to "Authentication Service (/authn/login), Password Service (/password/verify)" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

Responses

Refer to "Authentication Service (/authn/login), Password Service (/password/verify)" in the [AccessMatrix REST API Specification](#) for all possible responses.

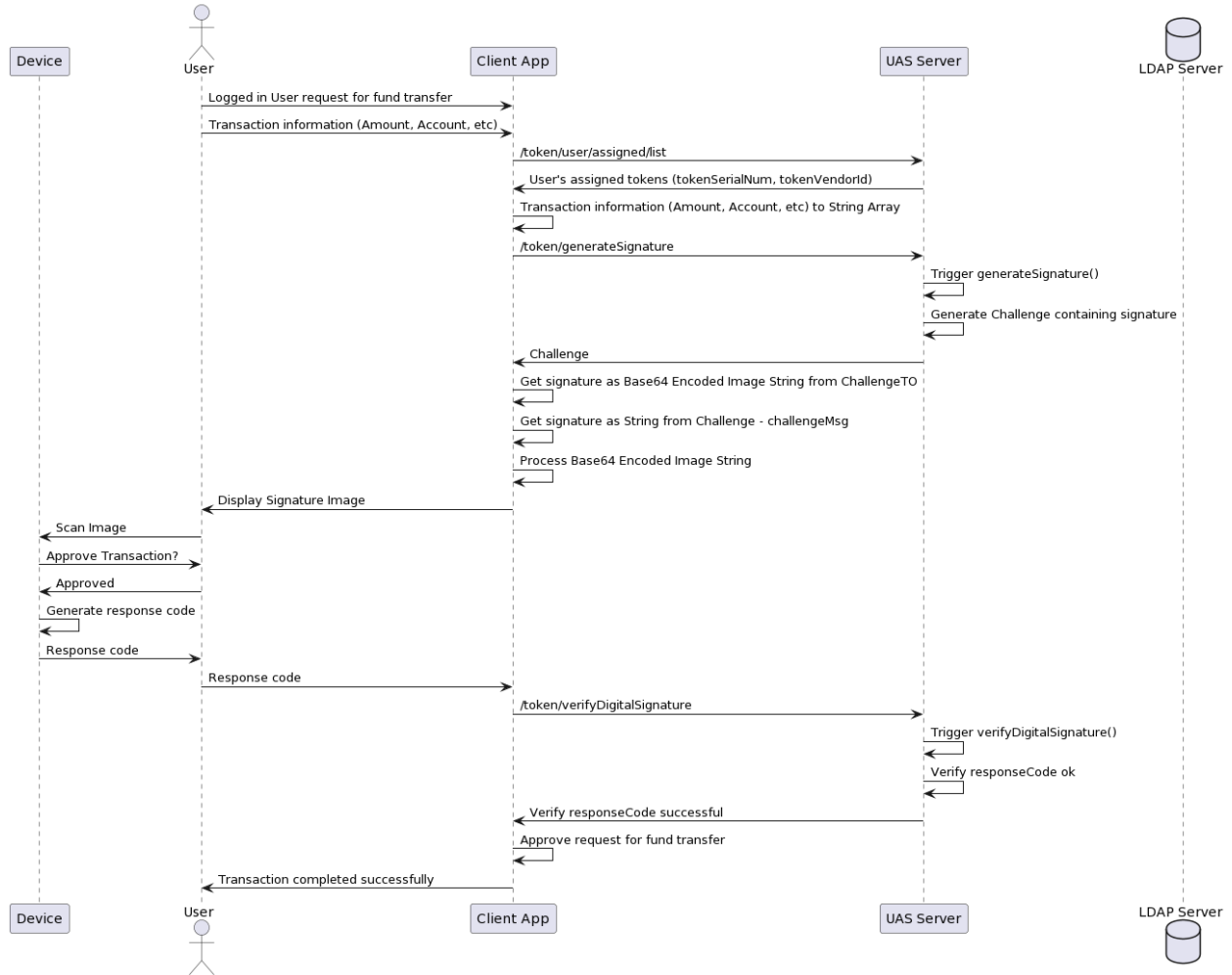
4.5. Standard Digital Signature Verification

Sequence Diagram

The sequence diagram below shows the sample use case of Digital Signature where a logged-in user tries to perform a transaction.

1. The client application generates a signature transaction message based on the transaction information keyed in by the user.
2. The client application triggers `/token/generateSignature` to obtain the server-generated signature message based on the user's assigned token.
3. The generated signature transaction message is returned as clear String and Base64 Encoded Image String to the client application.
4. The client application processes and displays the Base64 Encoded Image String to the user.
5. The user scans the image using their token device.
6. The token device prompts the user whether to sign this transaction; if the user authorizes the transaction, a response code will be generated.
7. The response code is then verified using `/token/verifyDigitalSignature`.
8. `verifyDigitalSignature` results are returned to the client application (success/fail).

OneSpan Digital Signature Sample Use Case Sequence Diagram



REST Sample Code

Signature Generation	
URL: /token/generateSignature	
Request	
JSON	
<pre> { "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "seeds": ["A01545490", "100.05", "832"] } </pre>	

Signature Generation

Response

JSON

```
{
  "result": {
    "params": {
      "imageBase64":
"0002nEZMqgYDmlfsYwRvzi_QSV_iOzASliem....TvNGKd2mGTyhvBRkA4rVwQidCD4Noe7FHd6
MrvKZ-__rk",
    }
  }
}
```

Signature Verification

URL: /token/verifyDigitalSignature

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "seeds": [
    "A01545490",
    "100.05",
    "832"
  ],
  "signature": "654923"
}
```

Response

JSON

```
{
  "result": {
    "params": {
      "retCode": "0",
      "deviceType": "Android",
      "deviceFriendlyName": "MyToken",
      "instanceNumber": "1"
    }
  }
}
```

Possible Parameters & Exceptions

Refer to "Token Service (/token/generateSignature, /token/verifyDigitalSignature)" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

Responses

Refer to "Token Service (/token/generateSignature, /token/verifyDigitalSignature)" in the [AccessMatrix REST API Specification](#) for all possible responses.

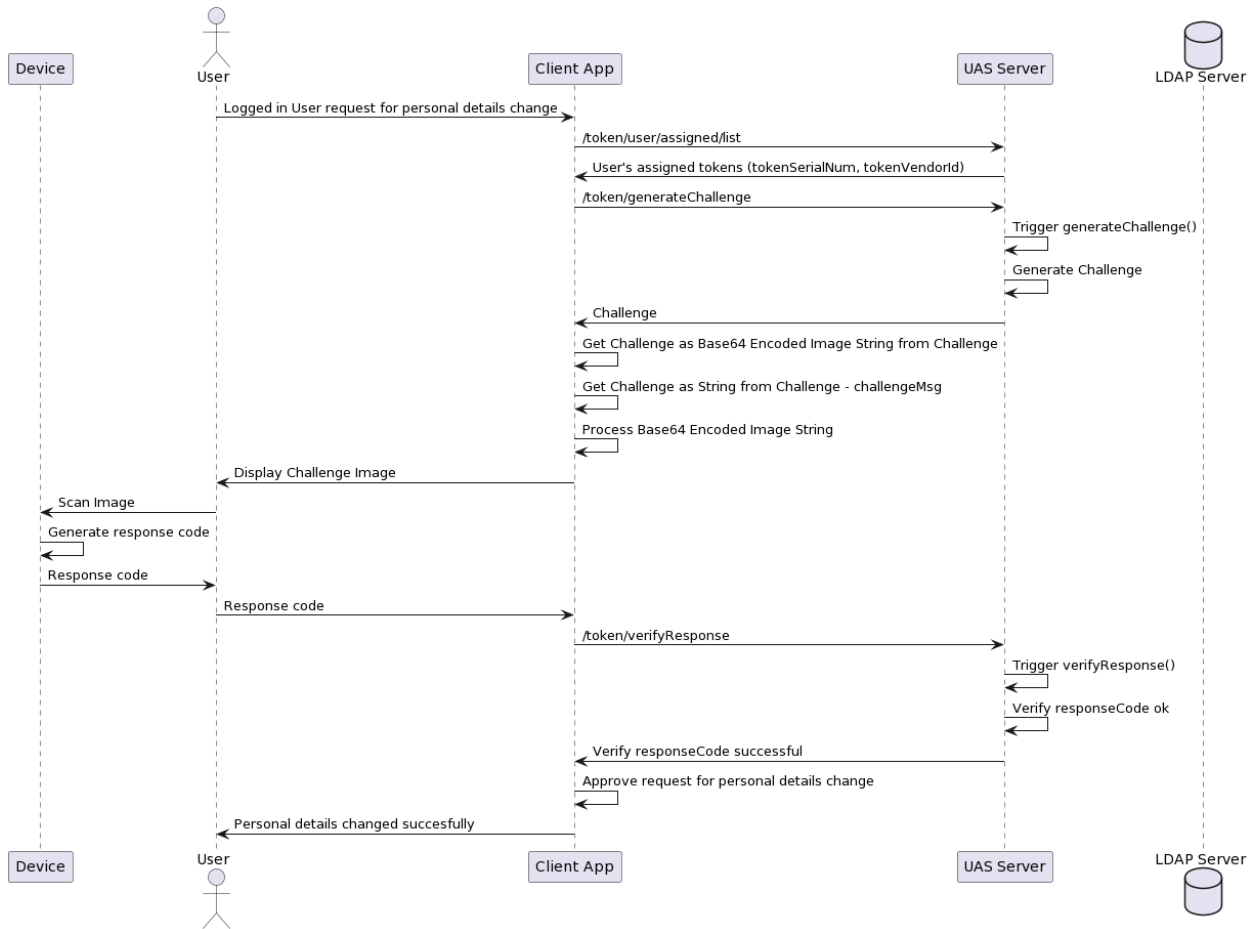
4.6. Other Challenge-Response Sample

Sequence Diagram

The sequence diagram below shows the sample use case of Challenge-Response where a logged-in user tries to request personal details change.

1. The client application usually challenges the user before the request is being approved.
 2. The client application triggers `/token/generateChallenge` to obtain the server-generated challenge based on the user's assigned token.
 3. The generated challenge is returned as clear String and Base64 Encoded Image String to the client application.
 4. The client application processes and displays the Base64 Encoded Image String to the user.
 5. The user scans the image using their token device to generate a response code.
 6. The response code is then verified using `/token/verifyResponse`.
 7. `verifyResponseBySerialNum()` results are returned to client application (success/fail).
-

OneSpan Challenge-Response Sample Use Case Sequence Diagram



REST Sample Code

Challenge Generation	
URL: /token/generateChallenge	
Request	
JSON	
<pre>{ "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", }</pre>	
Response	
JSON	
<pre>{ "result": { "sessionToken": "10001hUX9....0LtRMKes", } }</pre>	

Challenge Generation
<pre> { "challenge": { "challengeToken": "03F96D794431415130323031....3764D6C0557C843734C7B88EDF80" "params": { "imageBase64": "0002nEZMqgYDmlfsYwRvzi_QSV_iOzASliem....TvNGKd2mGTyhvBRkA4rVwQidCD4Noe7FHd6 MrvKZ-__rk", "challenge": "03F96D794431415130323031....3764D6C0557C843734C7B88EDF80" } } } </pre>
Verify Response Verification
URL: /token/verifyResponse
Request JSON <pre> { "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "password": "654923", "challenge": "03F96D794431415130323031....3764D6C0557C843734C7B88EDF80" } </pre>
Response JSON <pre> { "result": { "params": { "retCode": "0", "deviceType": "Android", "deviceFriendlyName": "MyToken", "instanceNumber": "1" } } } </pre>

Possible Parameters & Exceptions

Refer to "Token Service (/token/generateChallenge /token/verifyResponse)" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

Responses

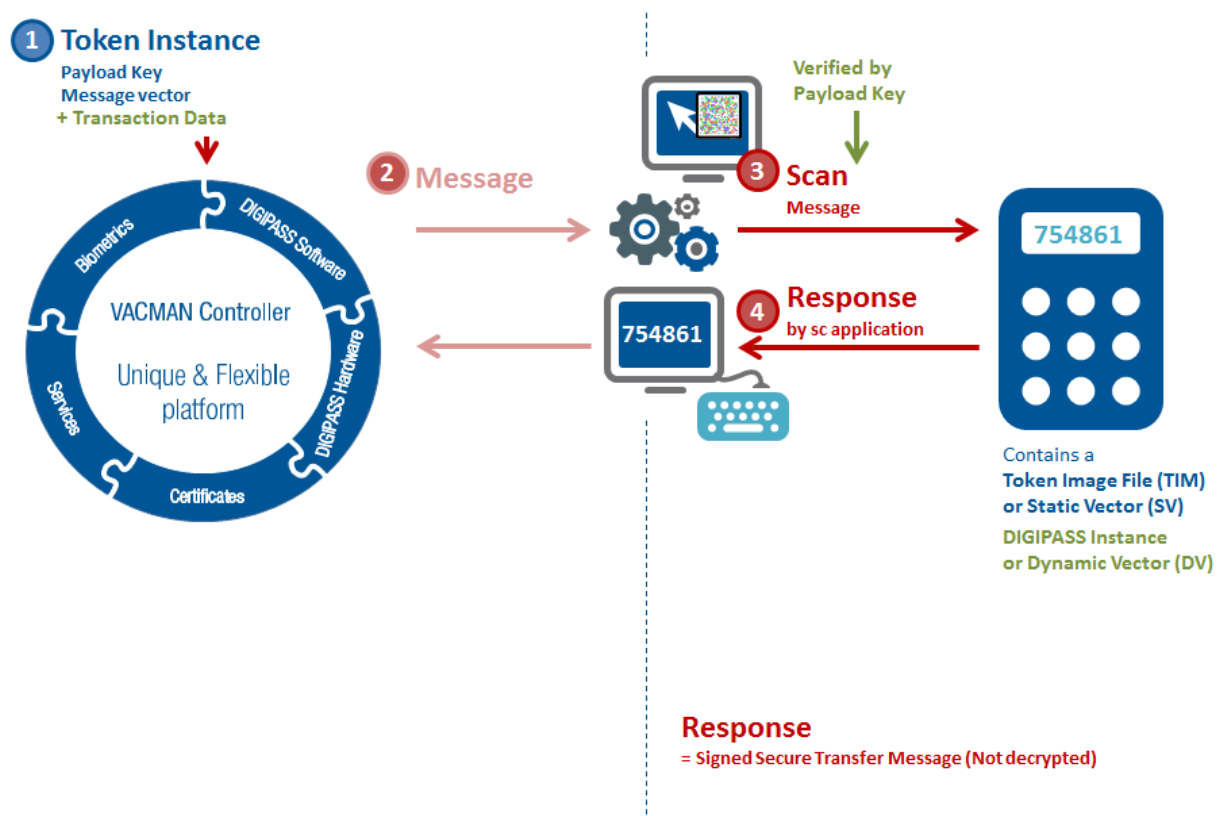
Refer to "Token Service (/token/generateChallenge /token/verifyResponse)" in the [AccessMatrix REST API Specification](#) for all possible responses.

4.7. Secure Channel Verification

This section explains about the new feature introduced by OneSpan for OneSpan Multi-Device Concept.

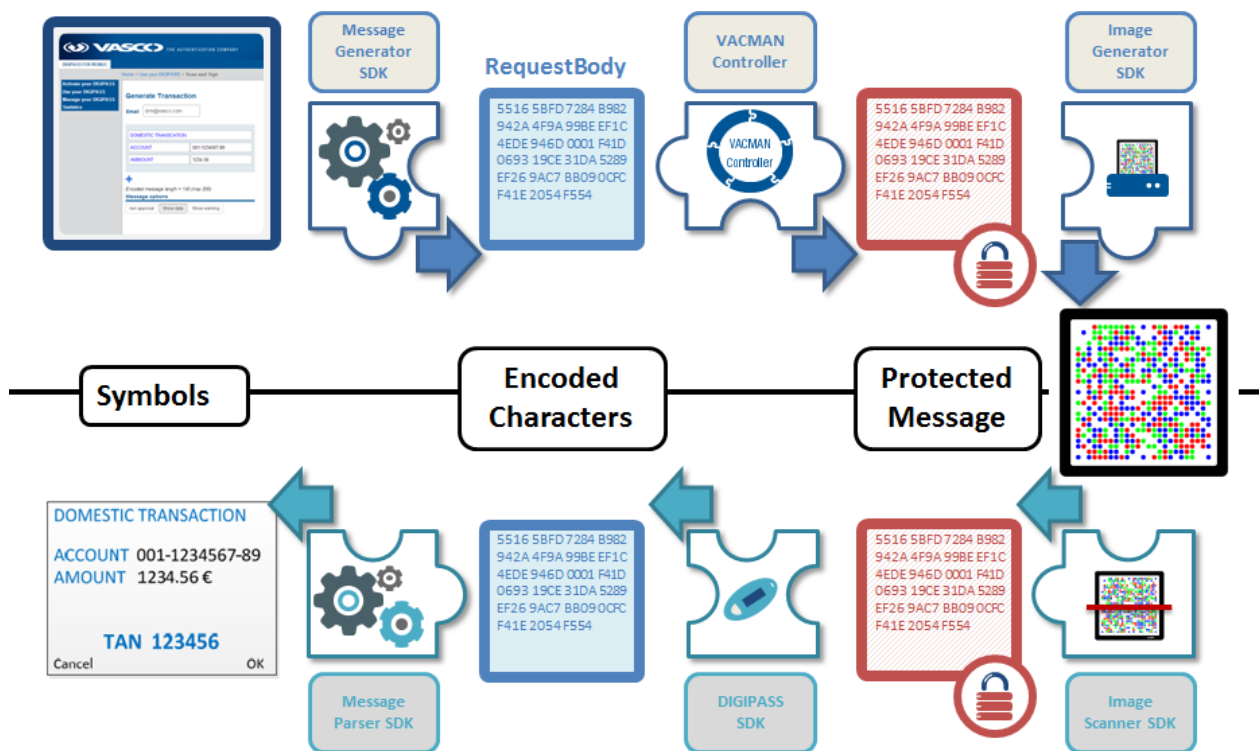
- Secure channel feature is known to be a more secure version of standard Digital Signature.
- The concept of a secure channel is that the generated signature message is protected by payload key in the server and is only readable when scanned by the token device's which has the same payload key.
- The payload key is unique for each token and is loaded into the token device once it is activated.

Flow of secure channel message generation, scanning and response generation:



Secure Channel Processing Flow:

The diagram below shows how the secure channel message is generated and protected, and how the device obtains and read the protected secure channel message before displaying the information to the user.



This section shows how the secure channel feature is being used to perform a secure transaction signing.

1. The user enters the transaction data as usual and the server will use the data to generate a secure channel message.
2. This secure channel message contains the transaction data and is protected whereby only the user's token device able to read.
3. User's device reads the protected secure channel message after scanning the CrontoSign image of the message and display it to the user for further action.
4. If the user authorizes the transaction, a response code will be displayed and the user needs to enter the response code back to the server.
5. The response code is a Signed Secure Transfer Message (Not decrypted) which can only be decrypted by the server.

Sequence Diagram

The example use case is when a user request for a fund transfer (assuming the user has already been authenticated).

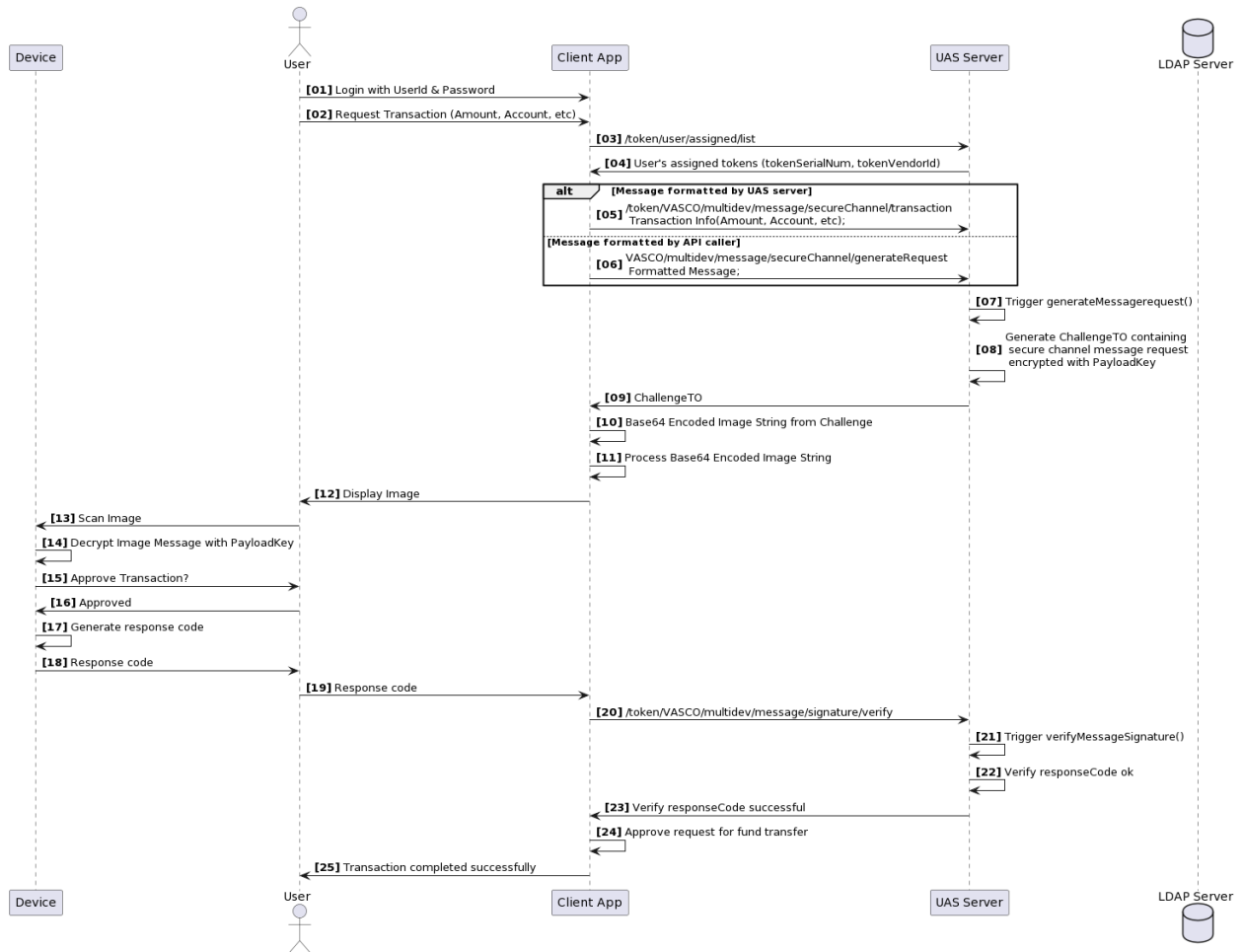
-
1. Transaction information is entered (Amount, Account, etc) by the user as prompted by the client application.
 2. The client application transforms the transaction information into a map (msgContent).
 3. The client application calls /token/user/assigned/list API to retrieve the user's assigned token.
 4. The client application calls /token/VASCO/multidev/message/secureChannel/transaction to generate the secure channel message for that particular transaction.



Note: If the client application is passing in its own pre-formatted message values, it will call /token/VASCO/multidev/message/secureChannel/generateRequest to directly generate the secure channel message.

- "msgRequestType" refers to the type of secure channel message request. Currently, "TransactionSigning" or "FreeText" is supported. In this example use case, "TransactionSigning" will be used.
 - "msgTitle" refers to the title of the secure channel message. This title will be displayed to the user, e.g. the title can be "Fund Transfer", "Transaction Limit", etc.
 - "msgContent" refers to the transaction information which has been transformed into a JSON object by the client application.
5. After calling /token/VASCO/multidev/message/secureChannel/transaction, an image string encoded in base64 will be returned to client application.
 6. Client application processes and displays the image to the user. User scans and authorizes the transaction to obtain the response code
 7. Response code is verified by /token/VASCO/multidev/message/signature/verify.
-

OneSpan Secure Channel Sample Use Case Sequence Diagram



REST Sample Code

Secure Channel Transaction	
URL: /token/VASCO/multidev/message/secureChannel/transaction	
Request	
JSON	
<pre> { "sessionToken": "10001hUX9....0LtRMKes", "vendor": "VASCO_MULTI_DEVICE", "serialNumber": "0123456", "title": "Transfer money", "content": { "Acc": "P08540123", "Amount": "18.99", "Purpose": "Tips" } } </pre>	

Secure Channel Transaction

Response

JSON

```
{
  "result": {
    "params": {
      "imageBase64":
"0002nEZMqgYDmlfsYwRvzi_QSV_iOzASliem....TvNGKd2mGTyhvBRkA4rVwQidCD4Noe7FHd6
MrvKZ-__rk",
      "signedMessage":
"03F96D794431415130323031....3764D6C0557C843734C7B88EDF80"
    }
  }
}
```

Request Generation

URL: /token/VASCO/multidev/message/secureChannel/generateRequest

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "message": "Formatted_Message"
}
```

Response

JSON

```
{
  "result": "00C1C3E489C9711E81D0797C19A8...421131047F14E570",
  "amProcessingTimeMillis": 345
}
```

Signature Verification

URL: /token/VASCO/multidev/message/signature/verify

Request

JSON

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
}
```

Secure Channel Transaction	
	<pre> "serialNumber": "0123456", "message": "03F96D794431415130323031...3764D6C0557C843734C7B88EDF80", "responseCode": "854023", } </pre>
Response	
	JSON <pre> { "result": { "params": { "retCode": "0", "deviceType": "Android", "deviceFriendlyName": "MyToken", "instanceNumber": "1" } } } </pre>

Possible Parameters & Exceptions

Refer to "Token Service (/token/user/assigned/list, /token/VASCO/multidev/message/secureChannel/transaction, /token/VASCO/multidev/message/signature/verify)" in the [AccessMatrix REST API Specification](#) for all possible parameters and exceptions.

Responses

Refer to "Token Service (/token/user/assigned/list, /token/VASCO/multidev/message/secureChannel/transaction, /token/VASCO/multidev/message/signature/verify)" in the [AccessMatrix REST API Specification](#) for all possible responses.

4.8. Push Notification Feature



Note: UAS only supports push notification registration. The application should rely on their push gateway or any existing infrastructure for push notification handling.

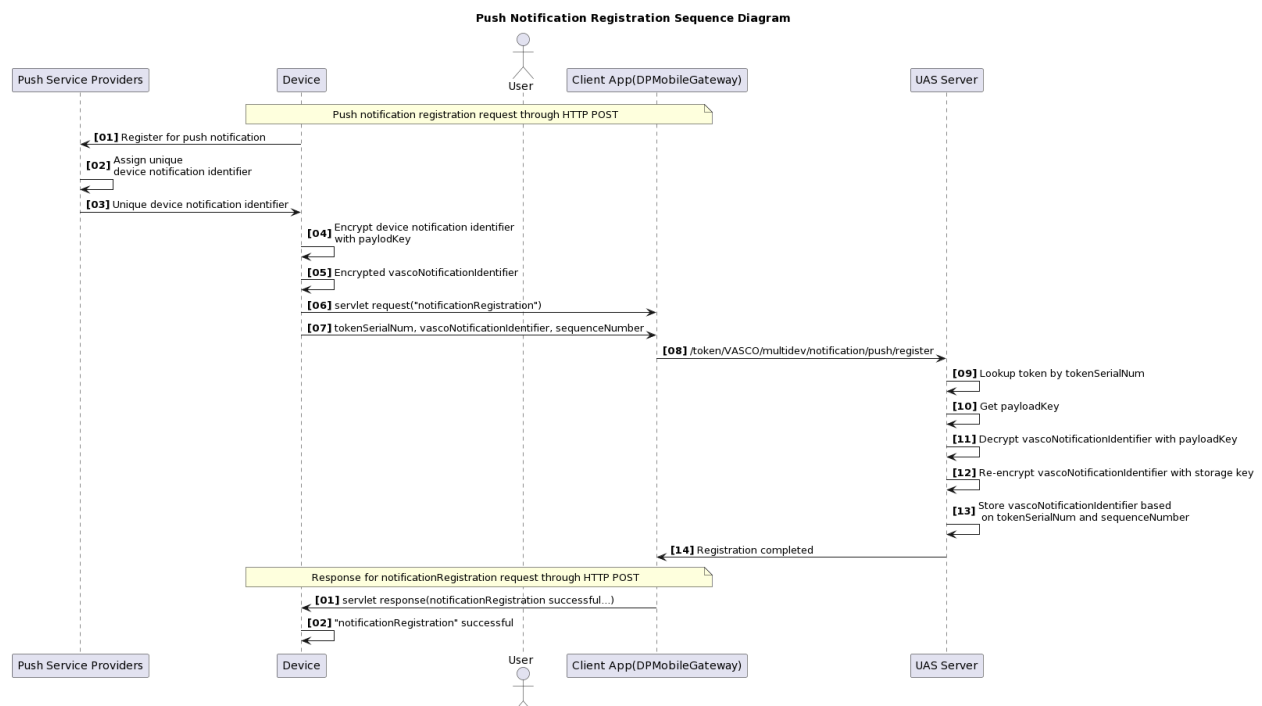
Push Notification Registration

Push notification registration is a feature in UAS to allow the application to register and store the push notification ID of each user device (instance).

1. Configuration on **DIGIPASS.xml** is required to enable push notification registration before generating the mobile app.
 2. Push notification registration is invoked automatically immediately after successful online activation.
 3. Before initiating the registration, the mobile app first automatically attempts to register & obtain a push notification identifier from the service providers (Google/apple).
 4. After successful registration with service providers, the notification identifier is then encrypted with **payloadKey** (obtained during Online Activation process).
 5. The encrypted notification identifier is referred as **vascoNotificationIdentifier**.
 6. When invoked, the mobile app sends an **HTTP POST** request to the **DPMobileGateway** to start the push notification registration.
 7. The **HTTP POST** request contains the encrypted **vascoNotificationIdentifier** (encrypted with **payload key**), token instance **sequenceNumber** and token **serialNumber**.
 8. After receiving the **HTTP POST** request, the **DPMobileGateway** should call **/token/VASCO/multidev/notification/push/register** API to start the registration.
 9. Once this API is invoked, the **AccessMatrix UAS** server receives the encrypted **vascoNotificationIdentifier**.
 10. **AccessMatrix UAS** server decrypts the encrypted **vascoNotificationIdentifier** with **payload key** on the server.
 11. The clear **vascoNotificationIdentifier** is re-encrypted with server storage key before storing into the database.
-

Sequence Diagram for Push Notification Registration

This diagram shows how a mobile app initiates the push notification registration once Online Activation completed successfully. This process is done automatically in the background and is transparent to the user. Push notification registration may or may not be successful depending on whether: the mobile app has been enabled with push notification feature, the mobile able or not able to obtain a notificationIdentifier from push notification service providers, the connection from the mobile app to DPMobileGateway is stable or not, and other unforeseen constraints.



Sample Code for Push Notification Registration

This sample code is to be written in DPMobileGateway servlet and deployed in DMZ zone. The mobile app will then send an **HTTP POST** request to DPMobileGateway to invoke the sample code for push notification registration.

Push Notification Registration Sample Code	
URL: /token/VASCO/multidev/notification/push/register	
Request	
	JSON

Push Notification Registration Sample Code

```
{
  "sessionToken": "10001hUX9....0LtRMKes",
  "vendor": "VASCO_MULTI_DEVICE",
  "serialNumber": "0123456",
  "sequenceNumber": "1",
  "identifier": "3764D6C0557C843734C7B88EDF80"
}
```

Response

JSON

```
{
  "result": {
    "params": {
      "retCode": "0",
      "message": "Successful",
      "notificationIdentifier": "8ED57C83764D6C0437"
    }
  }
}
```

Sample DIGIPASS.xml - Push Notification Registration

This is a sample of the configuration in **DIGIPASS.xml** to enable the push notification registration. The DPMobileGateway URL must be specified correctly so that the mobile can make a successful **HTTP POST** request to the servlet for push notification registration. Notice the payload parameter which includes all those necessary values(vascoNotificationIdentifier, token instance sequenceNumber, token serialNumber) that the server required for push notification registration.

XML

```
<!-- ----- -->
<!-- NOTIFICATIONS -->
<!-- ----- -->

<Notifications>
  <view title="Transaction Data Signing">
    <Labels>
      <Label id="GetTDSListWaitMessage"
value="Retrieving pending transaction, please
wait..."/>
      <Label id="TransactionValidationWaitMessage"
value="Validating the transaction, please wait..."/>
      <Label id="TransactionRejectionWaitMessage"
value="Rejecting the transaction, please wait..."/>
      <Label id="ReceivedHostCode" value="Server
```

```

response: ">
    </Labels>
    <Dynamics>
        <Dynamic id="TDSList" class="menuItem"/>
        <Dynamic id="TSDetails"
class="resultGenerated"/>
    </Dynamics>
    <Buttons>
        <Button id="Accept" value="Accept"/>
        <Button id="Deny" value="Deny" />
    </Buttons>
    <Errors>
        <Error id="TDSListEmpty" value="No
transaction pending."/>
        <Error id="TDSInvalidDataFields" value="The
data fields contained in the transaction are
invalid."/>
        <Error id="TDSInvalidDataFieldsNumber"
value="The number of data fields in the transaction is
invalid."/>
    </Errors>
</view>
<NotificationRegistration>
    <URL method="POST"
value="HTTP://192.168.50.107:8080/dp4mobile-
cronto/DPMobileGateway">
        <PayloadParameter key="action"
value="notificationRegistration" />
        <PayloadParameter key="serialNumber"
value="%_SerialNumber_" />
        <PayloadParameter key="sequenceNumber"
value="%_sequenceNumber_" />
        <PayloadParameter
key="vascoNotificationIdentifier"
value="%_VascoNotificationIdentifier_" />
    </URL>
</NotificationRegistration>

```



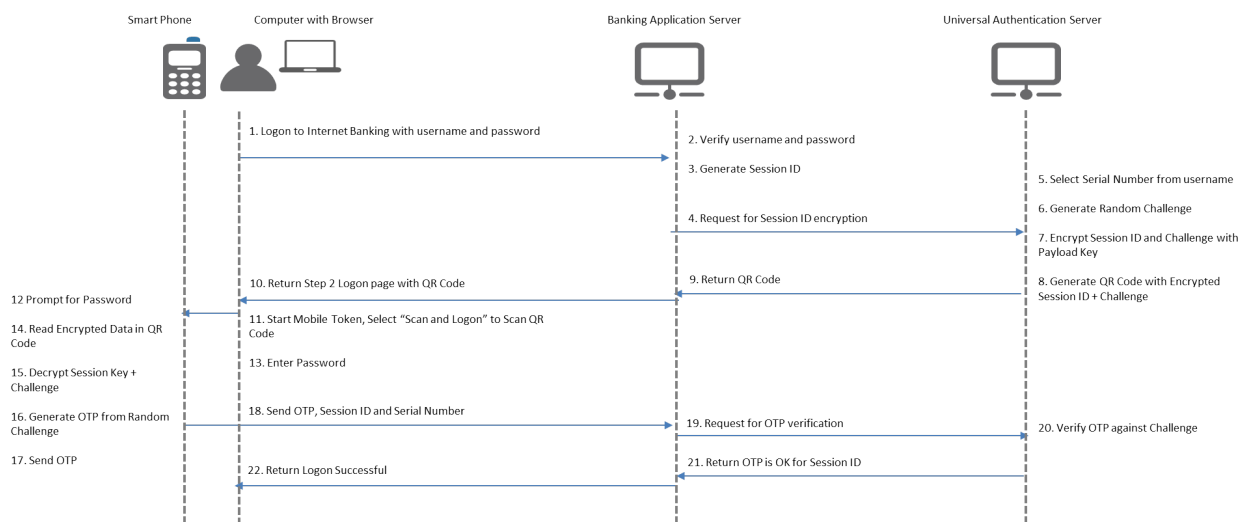
Note: The DPMobileGateway Servlet URL is
 "HTTP://192.168.50.107:8080/dp4mobile-cronto/DPMobileGateway".

5. UAS OneSpan Multi-Device Token Common Use Cases

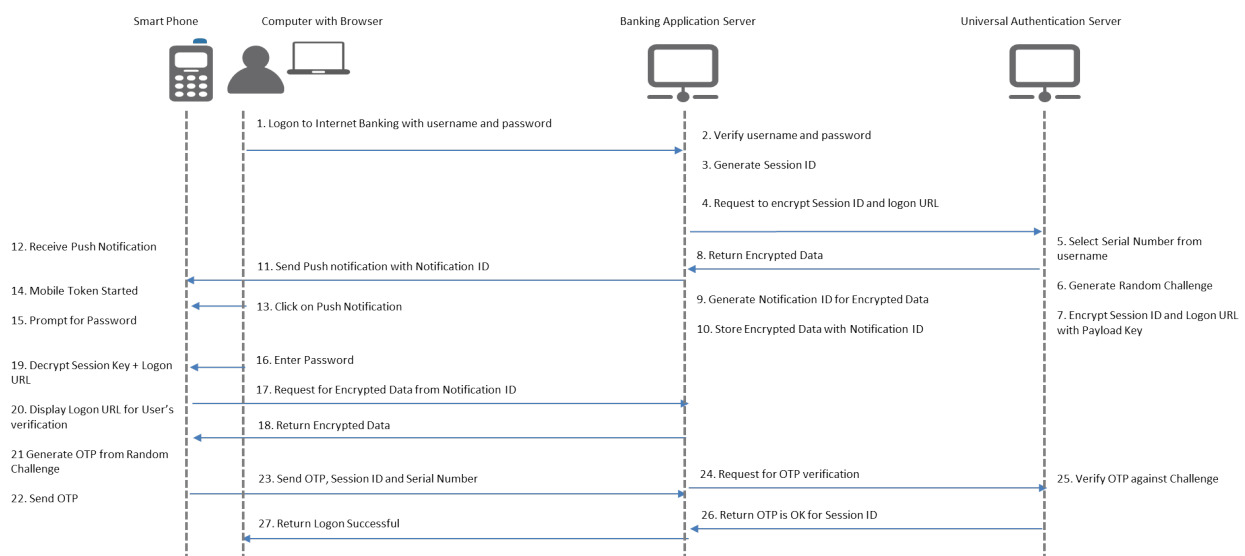
Scan and Login & Push and Login

1. The login request is either pushed to the user's mobile app through notification server or user scan the QRCode/CrontoSign for the login request.
2. The mobile app generates the OTP and sends an **HTTPS POST** request to the client application to authorize the login.
3. The client app receives the authorization and verifies the OTP with the AccessMatrix server.

Strong Authentication – Scan and Login



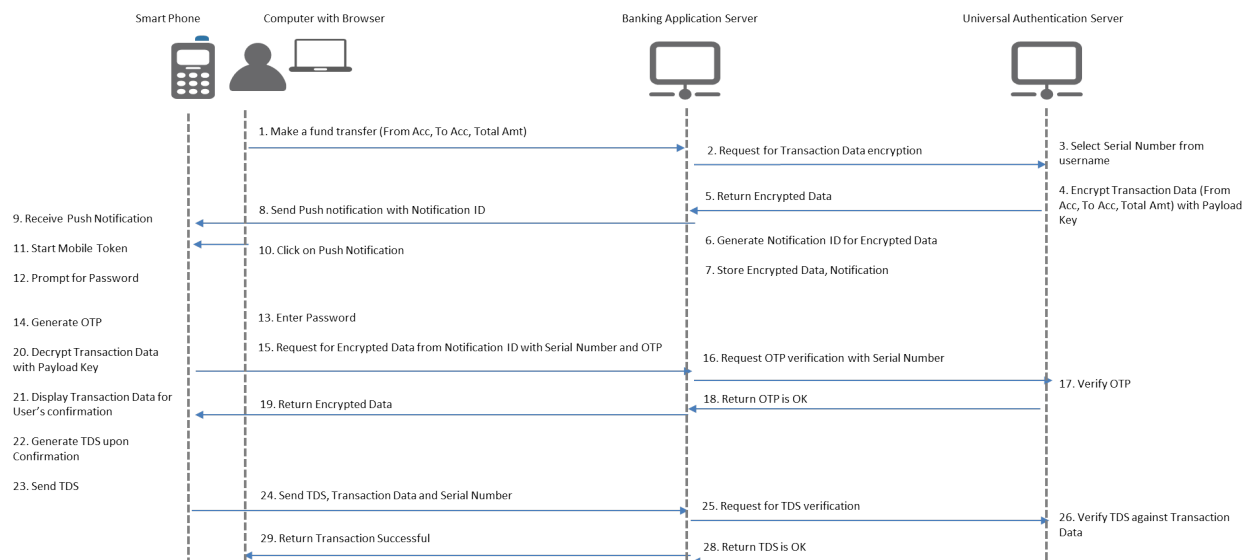
Strong Authentication – Push and Login



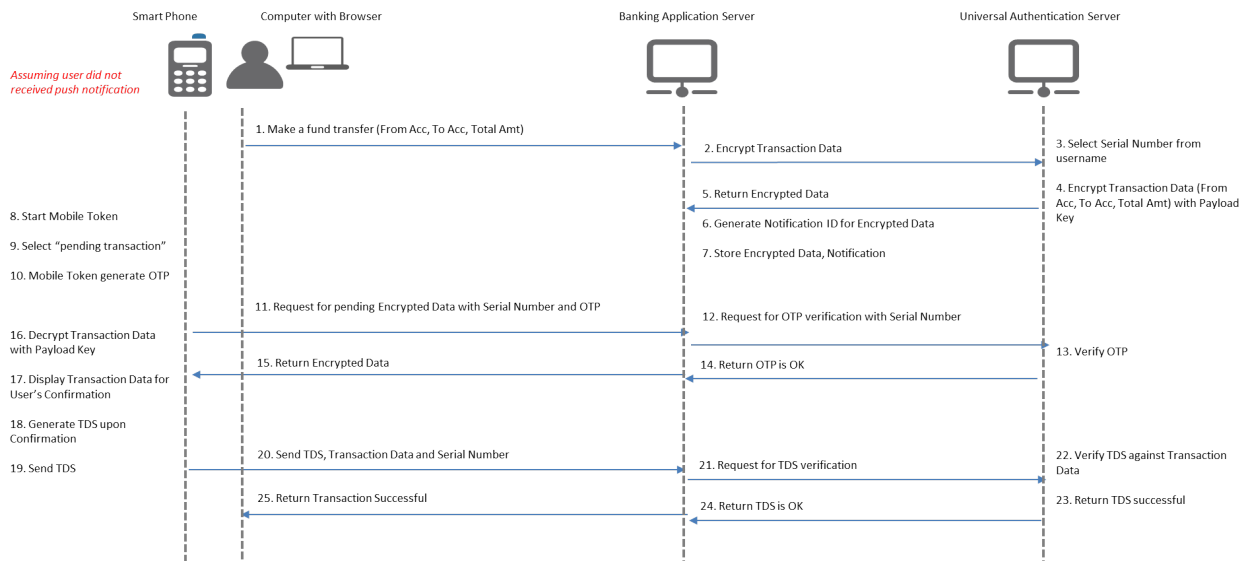
Push and Sign

1. The transaction is created and stored in the client's application. A notification is pushed to a mobile app through a notification server.
2. The mobile app receives the notification and retrieves the pending transaction from client app through an **HTTP GET** request.
3. The mobile app then generates the Signature Response upon the user's authorization and sends an **HTTPS POST** request to the client application to authorize the transaction.
4. The client app receives the authorization and verifies the Signature Response with the AccessMatrix server.

Transaction Signature – Push and Sign



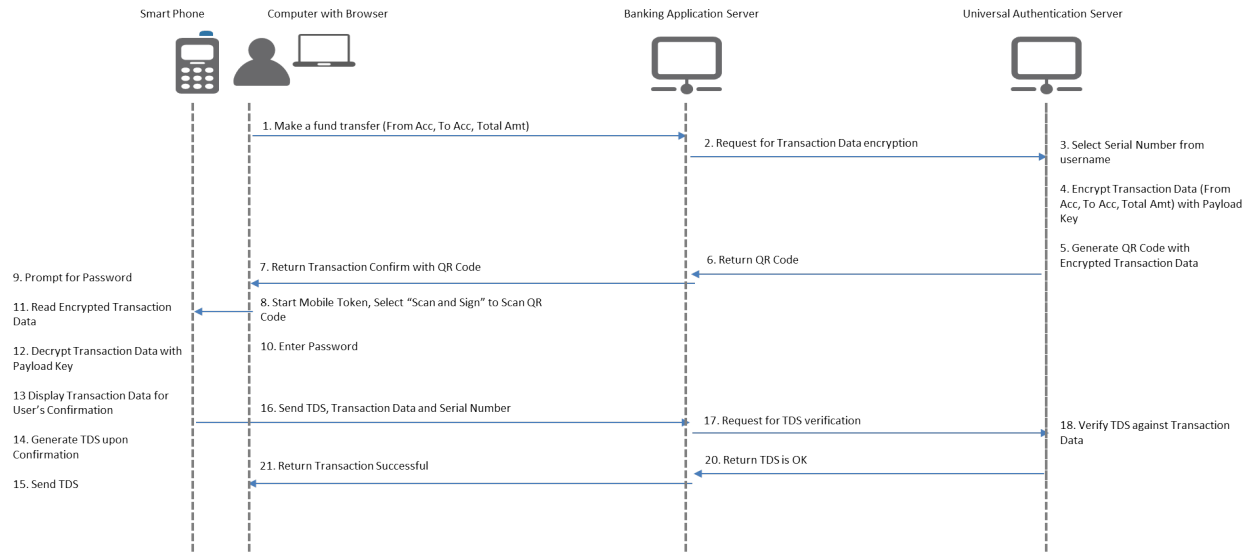
Transaction Signature – Direct Connect



Scan and Sign

1. The transaction is created and stored in the client's application.
2. The mobile app scans the QRCode/CrontoSign.
3. The mobile app reads the transaction data and generates a response signature upon user approval.
4. The user either manually keys in the response to the client application for verification or the mobile app sends an **HTTPS POST** request to the client application to authorize the transaction.
5. The client app receives the authorization and verifies the Signature Response with the AccessMatrix server.

Transaction Signature – Scan and Sign



6. Appendices

A. DPX Security and Protection

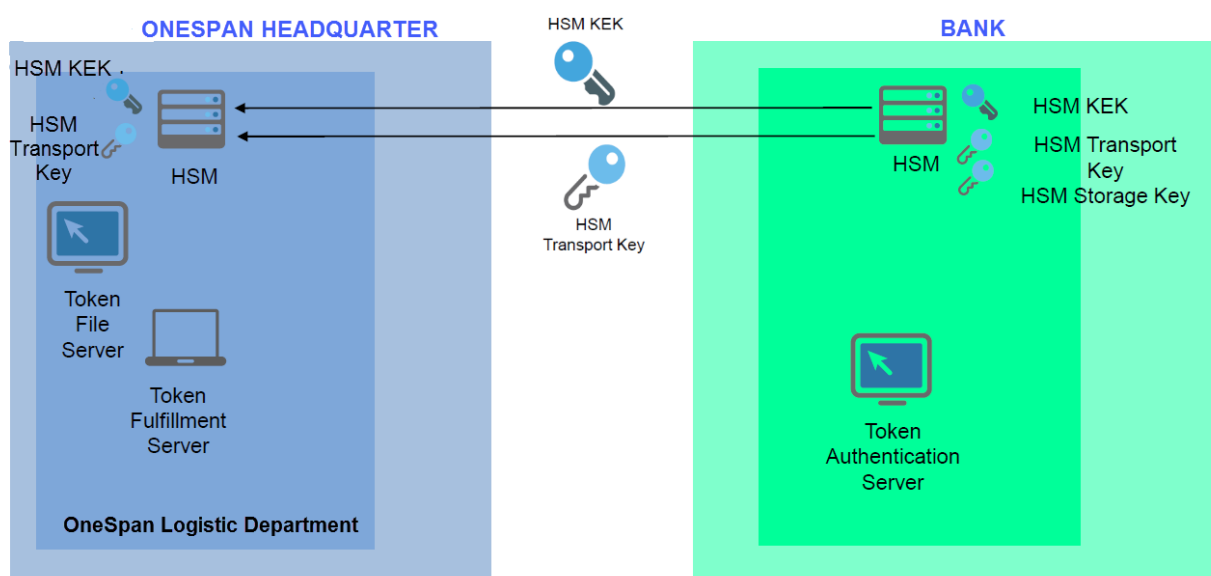
This section explains the DPX security protection during the exportation of Digipass Key by OneSpan. It includes the graphical explanation of key exchange ceremony between OneSpan and the bank; the double encrypted Digipass Key exported to DPX file. It also covers how the secure Digipass Key is used during import/usage process and how HSM protects the Digipass Key.

Exchange of HSM Keys

OneSpan and Bank will exchange their HSM keys before OneSpan can proceed with DPX Key generation based on the exchanged keys.

The illustration below shows:

1. The HSM KEK is generated in the bank's HSM and exchanged with OneSpan's HSM.
2. The HSM Transport Key is generated in the bank's HSM, wrapped with the KEK, and then exchanged with OneSpan's HSM.

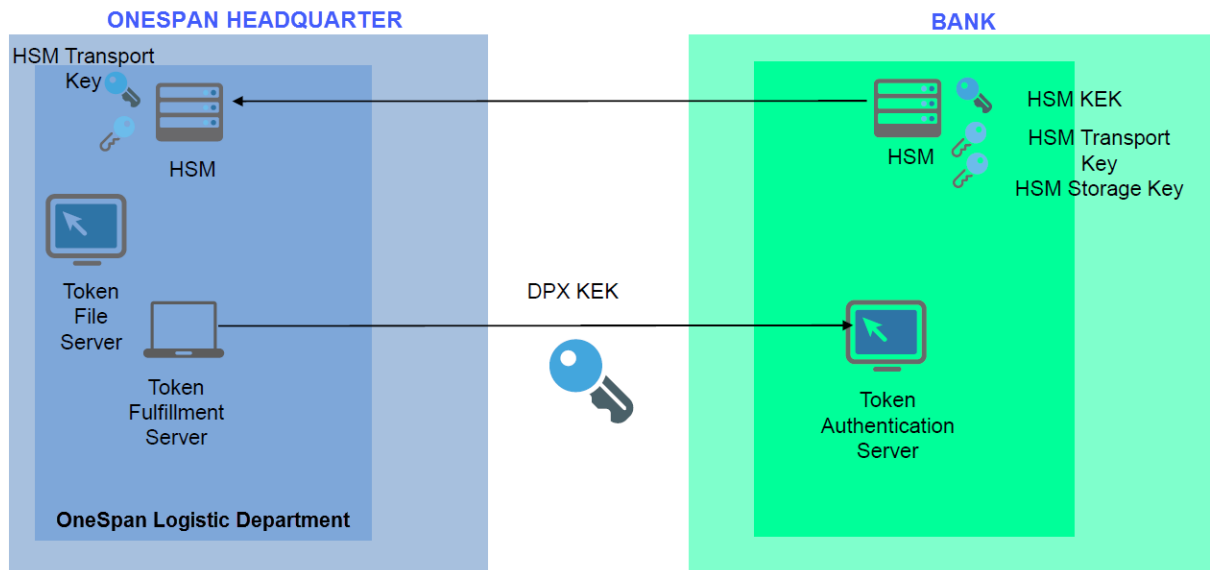


Exchange of Digipass Key

The DPX Key is generated by OneSpan and sent to the bank. This DPX key will be used during the token import process.

The illustration below shows:

-
1. The DPX Key is generated by OneSpan (per token batch) and then sent to the bank using a PIN mailer.

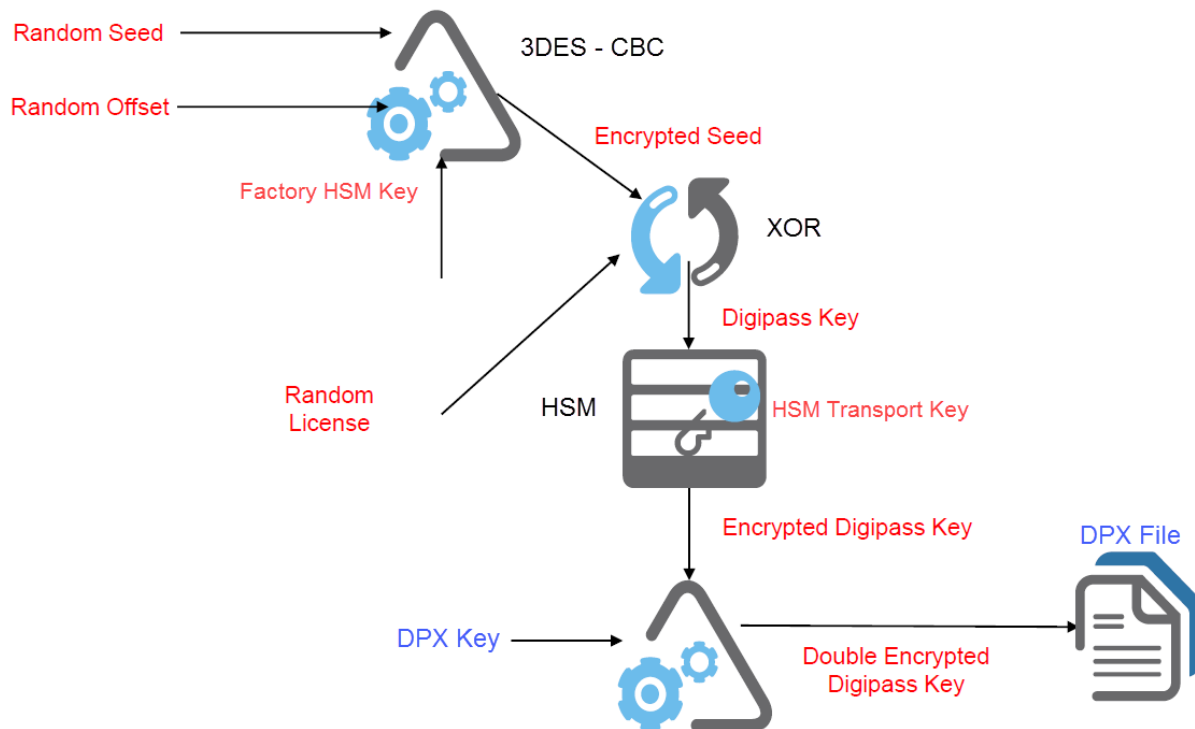


Exportation of Digipass Key to DPX File

Digipass Key (token seed in the DPX file) is generated by OneSpan using the same seed, offset and license used during the hardware token production. Then by having the HSM Transport Key (obtained during key exchange ceremony) and the DPX Key (generated after the key exchange ceremony), OneSpan performs Double Encryption on the Digipass Key before exporting the Digipass Key into DPX file. This DPX file will be used during the token import process.

The illustration below shows:

1. DIGIPASS key is created using the same random seed, random offset and random license.
 2. DIGIPASS key is encrypted by the HSM transport key before it is encrypted by the DPX key.
 3. The double-encrypted DIGIPASS is exported from the DPX file.
 4. The DPX file is printed in the PIN mailer.
-



Summary of DPX Double Encryption and DPX Delivery

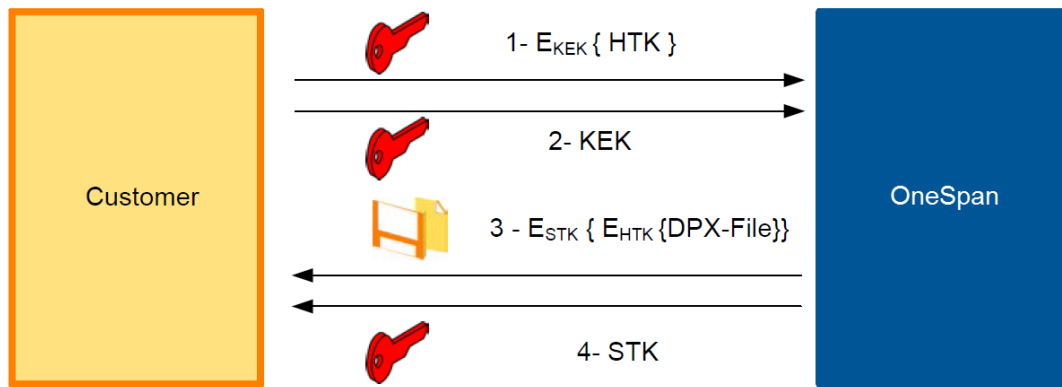
The illustration below shows how DPX double encryption and delivery are carried out from the customer to OneSpan, and vice versa.

1. The customer generates the HSM-level DPX transport key and sends it to OneSpan, encrypted under a Key Encryption Key (KEK).
2. Next, the customer sends the KEK to OneSpan.
3. OneSpan generates the software-level DPX transport key, and also generates and double-encrypts the DPX file before sending the DPX file to the customer.
4. OneSpan sends the software-level DPX transport key to the customer.



Notes:

- **HTK:** HSM level DPX Transport Key
- **KEK:** Key Encryption Key
- **STK:** Software level DPX Transport Key



B. Secure Importation of Double Encrypted DPX

During the token import process, the double-encrypted DPX file is read by AccessMatrix UAS Server. The first layer of encryption is decrypted using the DPX key (software-based DPX transport key). After decrypting the DPX, AccessMatrix will send the encrypted Digipass Key (token seed; content obtained after decrypting the DPX's first layer encryption) into HSM. HSM will decrypt the encrypted Digipass Key using HSM Transport Key (hardware-based DPX transport key) and then re-encrypt it with HSM Storage Key before sending it back to AccessMatrix server for storage. The whole process of decryption and re-encryption in HSM ensure that the Digipass Key (token seed) is not exposed at all. The AccessMatrix server does not know the decrypted content of the Digipass Key.

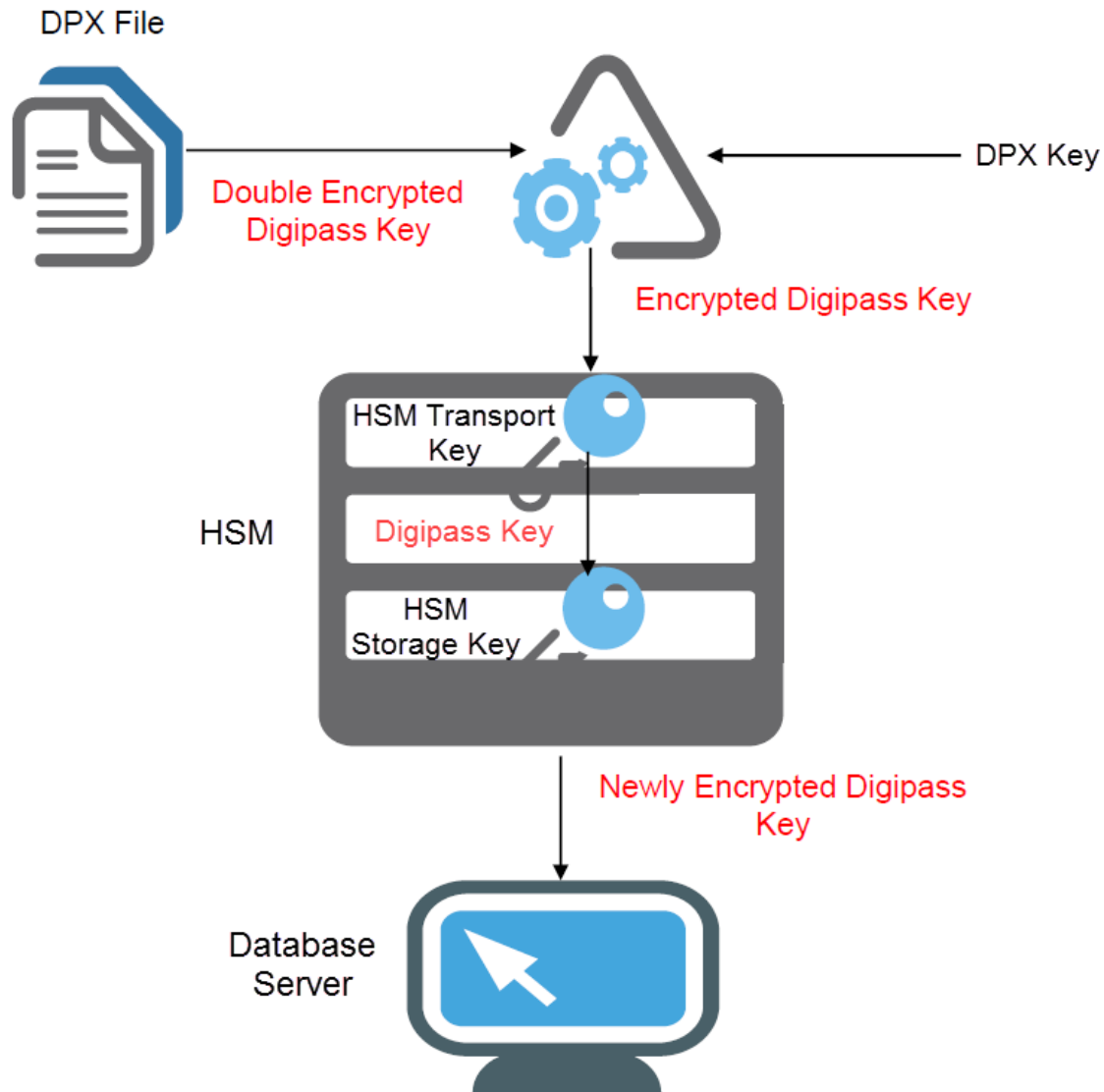
The illustration below shows how to secure a double-encrypted DFX file during import.

1. A double-encrypted DIGIPASS key is read from the DPX file before the DPX key decrypts it.
2. Next, the encrypted DIGIPASS key is decrypted by using the HSM transport key and then re-encrypted by using the HSM storage key.



Note: The DIGIPASS key is decrypted and re-encrypted within the HSM that will not leave the HSM.

3. The newly encrypted DIGIPASS key is stored in the database server.
-



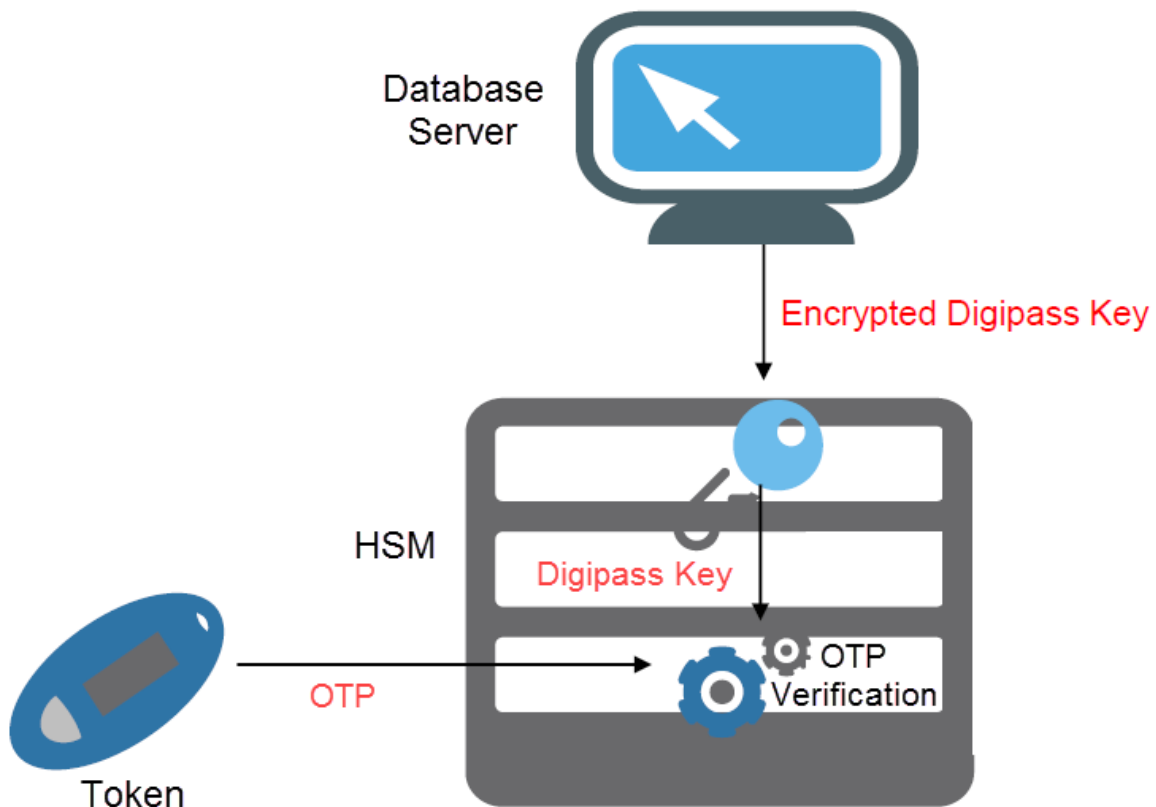
C. Secure Usage of Digipass Key (token seed)

HSM fully secures the usage of the Digipass Key. The decryption and usage are always performed inside the HSM. AccessMatrix server reads the encrypted Digipass Key from the database and sends it into the HSM together with the OTP for verification. The HSM then decrypts the encrypted Digipass Key using the HSM Storage Key. It also performs OTP verification using the decrypted Digipass Key, and returns the results to the AccessMatrix server.

The illustration below shows:

1. The encrypted Digipass key is read from the database server and decrypted in the HSM using the HSM storage key.
 2. The Digipass key is used within the HSM to verify the OTP.
-

Note: The Digipass key will not leave the HSM.

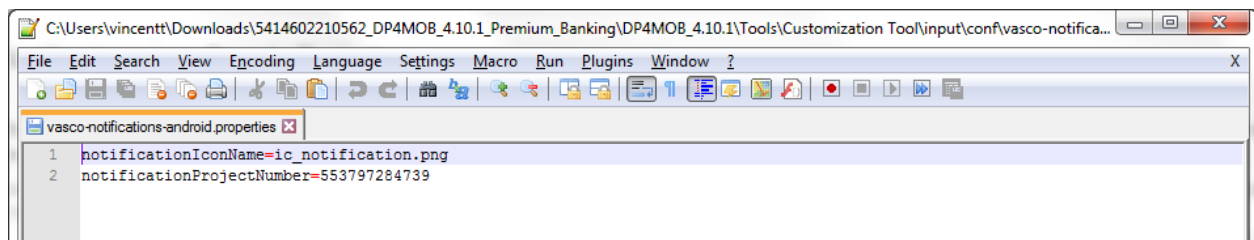


D. Enabling Push Notifications in Google/iOS Project

Enabling Push Notifications in a Google Project

To be able to send notifications, you need to adapt your Google project accordingly. In your developer console (<https://console.developers.google.com/project>), select your existing project. You will need your Project Number (available in the Overview Page) for the client (DPMobile Mobile APP) to register to Google's Cloud Messaging platform.

The Project Number must be specified in **vasco-notifications-android.properties** file under the DPMobile Customization Tool (*DP4MOB_4.10.1\Tools\Customization Tool\input\conf*) before the APK is generated. The following image is a sample of the **vasco-notifications-android.properties** file with the Project Number specified:



To enable push notifications in a Google Project:

1. Select **APIs & auth > APIs > Cloud Messaging for Android**.
2. Click **Enable API**.
3. In the pop-up window, click **Credentials**.
4. In the **Public API access** section, click **Create new Key**.
5. Read the information in the dialog box and click **Create**.
6. You will use the resulting API-KEY on the server-side to authenticate to the Google Cloud Messaging platform.
7. API-KEY is the androidProjectKey needed to be specified in the NotificationManager object defined in DPMobileGateway for push notification sending.

Enabling Push Notifications in an iOS Project

To use the push notification feature in your iOS application:

1. Connect to your Apple Member Center.
2. Locate or create your App ID that matches your bundle ID to enable the notification service. The bundle ID must be specific to your application (no wildcards are allowed for push notifications).
3. Generate a client SSL certificate for the connection between your notification server and the Apple Push Notification Service.

Note: You can work with separate client SSL certificates for development and production. When using a development certificate, the sandbox gateway must be used.

4. Verify that your provisioning profiles now include the Push Notification service.

For more information, refer to the Apple guidelines at

https://developer.apple.com/library/ios/documentation/IDEs/Conceptual/AppDistributionGuide/AddingCapabilities/AddingCapabilities.html#//apple_ref/doc/uid/TP40012582-CH26-SW11
